

PLC机考备考指南

- **PLC机考备考指南**
- **1 机试备考说明**
 - 1.1 环境说明
 - 1.2 题型说明
- **2 OCaml基本语法知识**
 - 2.1 基本数据类型和运算符
 - 2.2 let绑定
 - 2.3 函数
 - 2.3.1 如何理解函数是一等公民
 - 2.3.2 柯里化 (Currying)
 - 2.3.3 递归函数
 - 2.3.4 高阶函数
 - 2.3.5 函数语法糖
 - 2.4 if-else和模式匹配
 - 2.4.1 If-else
 - 2.4.2 模式匹配match...with
 - 2.5 列表
 - 2.6 元组
 - 2.7 变体
 - 2.7.1 变体的种类与定义
 - 2.7.2 变体的使用
 - 2.8 记录
 - 2.8.1 基本语法与使用：
 - 2.8.2 记录的更新
 - 2.8.3 记录双关语 (Record Punning)
 - 2.8.4 核心应用场景

2.9 元组、变体、记录的对比

2.10 引用

2.11 异常

2.12 模块

2.12.1 核心定义：模块 (Module) 与 签名 (Signature)

2.12.2 如何使用模块

2.12.3 函子 (Functor) —— 模块的“函数”

2.13 类

2.13.1. 核心定义：类 (Blueprint) vs. 对象 (Instance)

2.13.2 核心语法与对象创建

2.13.3 多态类 (Polymorphic Classes)

2.13.4 继承

2.13.5 抽象与封装

2.14 模块和类的对比

2.14.1 使用方式上的对比

2.14.2 对模块和类特性和作用的理解

2.14.3 为什么编译器项目用模块而不是类

• 3 Dune项目

3.1 dune项目结构

3.1.1 源代码目录

3.1.2 配置与生成文件

3.2 dune配置文件

3.2.1 dune-project

3.2.2 lib下的dune

3.2.3 bin下的dune

3.3 Dune相关命令操作

• 4 编译器前端

4.1 编译器前端的任务

4.1.1 前端工作流程

4.1.2 Top-down和Bottom-up方法

4.2 AST与文法

4.2.1 AST的概念

4.2.2 如何由文法设计AST?

4.2.3 从 AST 到文法

4.2.4 AST vs. CST (具体语法树)

4.3 ocamllex

4.3.1 .mll文件的结构

4.3.2 规则区

4.5 ocaml yacc (Bottom-up方法)

4.5.1 .mly文件的结构

4.5.2 Ocaml代码区

4.5.3 yacc声明区

4.5.4 语法规则区

4.6 手搓Parser (Top-down方法)

4.6.1 改造文法与定义 Token

4.6.2 实现Parser.ml

• 5 解释器 Interpreter

5.1 解释器的介绍

5.2 求值 (Evaluation)

5.2.1 小步求值(Small-Step Semantics)

5.2.2 大步求值(Big-Step Semantics)

5.2.3 对比总结

5.3 SimPL

5.3.1 SimPL 语法概览

5.3.2 替换 (Substitution)

5.3.3 求值规则 (Evaluation Rules)

5.3.4 实现参考

5.4 Lambda Calculus Interpreter

5.4.1 Lambda 演算的语法

5.4.2 Lambda 演算的求值语义 (Evaluation Semantics)

5.4.3 避免捕捉的替换 (Capture-Avoiding Substitution)

5.4.4 实现示例

6 附件

6.1 常见符号总结

6.2 大小写总结

6.3 常用数据模块库函数速查

6.3.1 List库函数速查表

6.3.2 String库函数速查表

6.3.3 类型转换函数表

6.3.4 Printf函数速查表

1 机试备考说明

1.1 环境说明

- 在VScode中用WSL编写代码，需要注意的点是两个系统的文件的互传
 - Win->Ubuntu：窗口拖拽
 - Ubuntu->Win：右键文件，用VScode的Download功能
- 助教说项目文件下发形式是通过机房系统统一发送到各个电脑，不是从希冀上下载。建议考试时先备份一份，免得自己改废了重新发送浪费时间

1.2 题型说明

相关dune文件和输入输出已经写好，只需关心具体内容

- 前三题：

- 纯算法题，不会考编译相关的知识，会下发整个dune项目方便调试输出
- 提交时注意文件名，以及题目要求的函数的签名
- 难度声称和tutorial中的练习差不多，有签到题也有难题
- 后三题：
 - 都在一个dune文件中完成，助教说尽量不让题目之间有相互依赖关系，免得第4题做不出来第5、6题也报废，但具体没法保证

考点预测：

- 前三题中递归函数、高阶函数、`let ... in...` 嵌套结构、`List` 操作、`match` 分支结构、根据题意写变体和Module需要重点复习
- 后三题：因为只有一个dune文件，为了考tutorial5的内容，大概率是解释器，不太可能只是一个编译器。对Ast.ml、lexer.mll和parser.mly的编写和两种求值方式、if和let语句的求值需要重点复习

2 OCaml基本语法知识

2.1 基本数据类型和运算符

基本数据类型：

- `int`：整数，如 `10`，`-5`
- `float`：浮点数，如 `3.14`，`2.0`
- `string`：字符串，如 `"hello"`
- `bool`：布尔值，`true` 或 `false`
- `char`：单个字符，如 `'a'`

基本运算：

- 算术运算：`+`，`-`，`*`，`/`（整数除法），`+.` ，`-.`，`*.`，`/.`（浮点数除法）
- 逻辑运算：`&&`（与），`||`（或），`not`（非）
- 比较运算：`=`（结构相等），`<>`（不等于），`>`，`<`，`>=`，`<=`

2.2 let绑定

功能：声明变量、定义作用域

- 绑定：
 - 全局绑定: `let <name> = <expression>`
 - 局部绑定: `let <name> = <expression1> in <expression2>`
- 解构数据：
 - 解构元组 (Tuple):

```
1 let my_tuple = (10, "Alice")
2 let (id, name) = my_tuple (* 直接把元组的两个元素分别绑定给了 id 和 name *)
```

- 解构记录 (Record):

```
1 type student = { student_id: int; student_name: string }
2 let alice = { student_id = 1; student_name = "Alice" }
3
4 let { student_id; student_name } = alice
```

- 进阶绑定
 - `let ... and ...`: 用于同时绑定多个（通常是互相递归的）函数或值

```
1 let rec is_even n =
2     if n = 0 then true else is_odd (n - 1)
3 and is_odd n =
4     if n = 0 then false else is_even (n - 1)
```

2.3 函数

2.3.1 如何理解函数是一等公民

- 函数拥有和其他数据类型（如 `int`, `string`）完全平等的地位
 - 把函数绑定到一个变量上: `let my_func = some_other_function`
 - 把函数作为参数传递给另一个函数
 - 让一个函数返回另一个函数作为结果

- 把函数存放在数据结构里（比如列表、元组）

2.3.2 柯里化（Currying）

柯里化：将接受多个参数的函数转换成一系列只接受单个参数的函数的技术，OCaml 中，是函数定义的默认行为

优势：

- 代码复用和特化
- 与高阶函数完美配合

```
1 let add x y = x + y
2 (* int -> int -> int *)
3 (* 只提供第一个参数 5 *)
4 let add_five = add 5 (* add_five 是一个新函数，类型是 int -> int *)
5 (* 使用这个特化的新函数 *)
6 let result1 = add_five 10 (* 结果是 15 *)
7
8 (*与元组函数进行对比*)
9 let add_tupled (x, y) = x + y
10 (* 类型: int * int -> int *)
11 (* 必须一次性提供所有参数，包裹在一个元组里 *)
12 let result = add_tupled (5, 10)
13 (* let add_five_tupled = add_tupled 5 (* 这会直接报错! *) *)
```

2.3.3 递归函数

递归函数 (`let rec`): 函数调用自己

```
1 (* 计算 n 的阶乘 *)
2 let rec factorial n =
3     if n = 0 then 1
4     else n * factorial (n - 1)
```

尾递归：递归调用本身就是整个函数的最后一步操作，不再等待下一层的结果，而是把当前的计算结果作为参数传递给下一层递归。

为什么尾递归重要：OCaml 编译器能够识别出尾递归，并将其优化成迭代 (iteration)，即循环。它不会像普通递归那样消耗调用栈空间，可以用来处理非常大的输入而不会导致栈溢出 (Stack Overflow)。

```
1 (* 使用尾递归计算 n 的阶乘 *)
2 let rec helper acc num =
3     if num = 0 then acc
4     else helper (acc * num) (num - 1)
```

2.3.4 高阶函数

高阶函数：接受函数作为参数或返回一个函数作为结果的函数，是函数式编程的标志性特征，能够让代码更简洁、更具表现力

```
1 let doubled_numbers = List.map (fun x -> x * 2) numbers
```

- `fun` 用来创建一个只用一次的匿名函数
- `List.map` 接收一个函数和一个列表，然后把这个函数应用到列表的每一个元素上

2.3.5 函数语法糖

语法糖让代码写起来更甜、读起来更顺

function 关键字： `let my_func = function | pattern1 -> ...` 是 `let my_func = fun x -> match x with | pattern1 -> ...` 的简写。

- 使用场景：当一个函数只接收一个参数，并且函数体第一件事就是对这个参数进行模式匹配时，使用 `function` 可以让代码更简洁。递归函数尤其常见。


```

1 (* 不使用 function 的写法 *)
2 let rec sum_list l =
3   match l with
4   | [] -> 0
5   | head :: tail -> head + sum_list tail
6
7 (* 使用 function 的简洁写法 *)
8 let rec sum_list_sugar = function
9   | [] -> 0
10  | head :: tail -> head + sum_list_sugar tail

```

管道操作符 `|>`： `x |> f` 等价于 `f(x)`，把左侧表达式的结果，作为参数传递给右侧的函数

- 优势：将深度嵌套的函数调用 `g(f(h(x)))`，转换成一条从左到右、清晰易读的数据处理流水线 `x |> h |> f |> g`

```

1 let result = [1; 2; 3; 4]
2   |> List.map (fun x -> x + 1) (* [2; 3; 4; 5] *)
3   |> List.filter (fun x -> x mod 2 = 0) (* [2; 4] *)
4   |> List.fold_left (+) 0 (* 2 + 4 = 6 *)
5 (* result 是 6 *)

```

2.4 if-else和模式匹配

特性	<code>if-else</code>	<code>match...with</code>
核心逻辑	基于 布尔值 (<code>bool</code>) 的真假判断	基于 任何类型 的 结构和值 的模式匹配
作用对象	条件表达式的结果必须是 <code>bool</code> 类型	可以是任何类型: <code>int</code> , <code>string</code> , <code>list</code> , <code>option</code> , 自定义变体等
数据提取	不支持。仅能根据条件选择分支。	非常强大。可以在匹配结构的同时, 将内部数据提取并绑定到新变量。
安全性	弱。只保证 <code>then</code> 和 <code>else</code> 分支类型一致, 编译器不检查逻辑是否完备。	强。编译器会进行 穷尽性检查, 如果未能覆盖所有可能的情况, 会直接报错, 有效防止运行时错误。
最佳场景	进行简单的二元或三元逻辑判断。	处理复杂数据结构 (如变体、列表)、需要从数据中提取值、确保代码逻辑覆盖所有情况时。

2.4.1 If-else

`if-else` 是一个 表达式 (Expression), 而不是语句。即它总会求值并返回一个结果

核心: 基于一个布尔值 (`bool`) 的判断, 在两个分支中选择一个

规则:

- 判断条件必须是 `bool` 类型

- 必须有 `else` 分支（只有返回类型是 `unit` 时，可以省略 `else`，它会隐式返回 `()`）
- `then` 和 `else` 分支的返回类型必须完全一致

嵌套 `if-else`

```
1 let describe_number n =  
2   if n > 0 then "正数"  
3   else if n < 0 then "负数"  
4   else "零"
```

2.4.2 模式匹配 `match...with`

核心：检查一个或多个值的“结构”或“形态”，并从中提取数据

优势：

- 可以匹配任何类型
- 解构数据：在匹配成功的同时，将数据的组成部分绑定到变量，无需额外操作

```
1 match some_option with  
2 | None -> ...  
3 | Some x -> (* 不仅匹配了 Some 结构，还把里面的值取出来绑定给了 x *)  
4   Printf.printf "The value inside is %d\n" x
```

- 安全性强：match 覆盖不完所有情况，编译器直接报错

进阶用法：

when 守卫 (Guard): 为一个模式附加额外的布尔判断条件。只有当模式匹配且 `when` 后面的条件为 `true` 时，该分支才会被选中

```
1 match some_tuple with  
2 | (x, y) when x = y -> "两个元素相等"  
3 | (x, y) -> "两个元素不相等"
```

as 别名: 在匹配一个复杂结构的同时，给这个结构本身起一个别名，方便后续使用

```

1 | match my_list with
2 | [] -> "Empty"
3 | (head :: _ as entire_list) -> (* 匹配非空列表，并给整个列表起别名
   entire_list *)
4 |     Printf.printf "The entire list has %d elements. The head is
   %d.\n"
5 |     (List.length entire_list) head

```

2.5 列表

不可变的、同质的序列，原型是递归变体

```

1 | type 'a list =
2 |   | [] (* 空列表, "nil" *)
3 |   | :: of 'a * 'a list (* 一个元素“连接”着另一个列表，读作 "cons" *)

```

原则：

- 同质性：列表中所有元素类型必须相同
- 不可变性：列表被创建后不能修改，增删改元素时，Ocaml会返回一个全新列表

基本操作：

- 创建列表：
 - 使用 `[]` 创建一个空列表
 - 使用 `[elem1; elem2; ...]` 直接创建包含元素的列表
- 组合列表：
 - `::` (cons) 操作符：将一个元素添加到列表的头部。效率高的 $O(1)$ 操作。
 - `@` (append) 操作符：将两个列表连接成一个新列表。效率低的 $O(n)$ 操作，需要遍历第一个列表的所有元素。

```

1 let empty_list = []
2 let nums = [2; 3; 4]
3
4 (* 使用 :: 将 1 添加到 nums 的头部 *)
5 let new_nums = 1 :: nums (* 结果是 [1; 2; 3; 4] *)
6
7 let list_a = [1; 2]
8 let list_b = [3; 4]
9
10 (* 使用 @ 连接两个列表 *)
11 let combined = list_a @ list_b (* 结果是 [1; 2; 3; 4] *)

```

核心用法：

列表的递归结构决定了处理它的最佳方式就是模式匹配和递归函数的结合。

- 模式匹配的两个基本形态：

1. `[]`：匹配空列表，通常作为递归的终止条件。
2. `head :: tail`：匹配任何非空列表，并同时将其解构为第一个元素 `head` 和剩余部分 `tail`（`tail` 本身也是一个列表）（也可以同时结构成多个元素和剩余部分，见练习）

```

1 let rec sum_list my_list =
2   match my_list with
3   | [] -> 0 (* 终止条件：空列表的总和是 0 *)
4   | head :: tail -> head + (sum_list tail) (* 递推步骤：总和 = 头元素
+ 剩余部分的总和 *)
5
6 (* 测试 *)
7 let my_sum = sum_list [1; 2; 3; 4] (* 结果是 10 *)

```

2.6 元组

- 将固定数量的元素，按照固定的顺序打包在一起
- 特点：
 - 大小固定，创建时有几个元素就是几个元素

- 类型异构，元组里的元素可以是不同类型
- 元组的定义

注意：真正创建元组的是逗号 `,` 操作符，而不是括号 `()`。括号的主要作用是分組和消除歧义，在不引起歧义的情况下可以省略括号，但是为了可读性，还是建议写括号。

```
1 (* 一个包含两个整数的元组，代表一个点的坐标 *)
2 let point = (10, 20)
3 (* 一个包含不同类型元素的元组，代表一个人的基本信息 *)
4 let person = ("Alice", 25, true) (* 姓名，年龄，是否为学生 *)
5
6 let print_sum (a, b) = print_int (a + b)
7
8 (* 正确的调用方式：将 (10, 20) 作为一个整体参数传递 *)
9 print_sum (10, 20)
10
11 (* 错误的调用方式 *)
12 (* print_sum 10, 20 *)
13 (* OCaml 会将其解析为 (print_sum 10), 20, 即先计算 print_sum 10,
14    然后试图将结果与 20 组成一个元组，这会导致类型错误。 *)
```

- 使用元组中的数据：解构

```
1 let point = (10, 20)
2 let (x, y) = point (* 将 point 拆开，10 绑定给 x，20 绑定给 y *)
3
4 Printf.printf "坐标是: (%d, %d)\n" x y
5
6 let person = ("Alice", 25, true)
7 let (name, age, is_student) = person
```

2.7 变体

核心概念：变体是“或” (OR), 根本目的是用来定义一个“可以是多种可能性之一”的类型。在类型理论中，变体也被称为和类型 (Sum Type) 或标签联合体 (Tagged Union)。

2.7.1 变体的种类与定义

- 简单枚举 (Simple Enums)

- 定义一组离散、互斥的常量或状态。
- 语法：构造器为不携带参数的标识符，首字母大写，用 `|` 分隔。

```
1 | type day_of_week = Monday | Tuesday | Wednesday | Thursday  
  | Friday | Saturday | Sunday
```

- 携带参数的变体 (Variants with Payloads)

- 表示一个事物可以是多种不同的结构，每种结构可以携带不同类型的数据。
- 语法：在构造器后使用 `of` 关键字来指定其携带的数据类型

```
1 | type payment_method =  
2 |   | CreditCard of string (* 携带卡号 *)  
3 |   | PayPal of string     (* 携带邮箱 *)  
4 |   | Cash of float        (* 携带金额 *)
```

- 参数化变体 (Parameterized Variants / Generics)

- 通过类型变量（如 `'a`）创建“泛型”变体，使其能够处理任何类型的数据，极大增强了代码的复用性。

```
1 | type 'a option =  
2 |   | None          (* 表示没有值 *)  
3 |   | Some of 'a    (* 表示有一个 'a 类型的值 *)
```

- 递归变体 (Recursive Variants)

- 变体的定义可以引用自身，用于构建任意复杂的递归数据结构，如链表、树等。

```
1 | type 'a list =  
2 |   | [] (* 空列表，基本情况 *)  
3 |   | (::) of 'a * 'a list (* 一个元素连接着另一个列表，递归情况 *)
```

2.7.2 变体的使用

核心用法：模式匹配

与变体交互最主要、最安全的方式就是模式匹配 (`match...with`)。变体定义了所有可能的情况，而模式匹配则强制你处理所有这些情况，否则编译器会报错。

```
1 let process_payment p =  
2   match p with  
3   | CreditCard card_number ->  
4     Printf.printf "正在处理信用卡: %s\n" card_number  
5   | PayPal email ->  
6     Printf.printf "正在通过 PayPal 账户: %s\n" email  
7   | Cash amount ->  
8     Printf.printf "收到现金: %.2f\n" amount
```

核心应用场景

- **构建抽象语法树 (AST)**: 这是变体在编译原理中最核心的应用。AST 本身就是一个递归变体，用来表示程序的结构。

```
1 type expression =  
2   | Number of int  
3   | Variable of string  
4   | Add of expression * expression (* 递归地包含自身 *)  
5   | Multiply of expression * expression (* 递归地包含自身 *)
```

- **定义状态机**: 清晰地表示一个系统可能处于的所有状态

```
1 type network_state =  
2   | Connecting  
3   | Connected of string (* 携带服务器 IP *)  
4   | Disconnected  
5   | Error of string (* 携带错误信息 *)
```

- **安全处理可能缺失的值**: 使用 `option` 类型是 OCaml 中避免 `null` 引用错误的标准做法

- 其他编程语言（如 Java, C#, Python, JavaScript）中，当你试图获取一个可能不存在的值时，函数可能会返回一个特殊值 `null`（或 `None`, `undefined`）。程序员在接收到返回值后，可能会忘记检查它是否为 `null`，而直接尝试使用它。容易导致程序在运行时突然崩溃，且这种错误难以在编译阶段发现。
- Caml 从语言层面彻底解决了这个问题，其核心思想是：用类型系统来明确表达“一个值可能不存在”这一可能性

```
1 type 'a option =  
2   | None          (* 表示“没有值”这个状态 *)  
3   | Some of 'a    (* 表示“有一个值”，并且这个值被包裹在 Some 构造器里 *)
```

- `None` 是一个简单的构造器，明确地代表了值的缺失。
- `Some` 是一个携带参数的构造器，它像一个“盒子”，安全地包裹着一个类型为 `'a` 的实际值

2.8 记录

核心概念：用来定义一个“必须同时包含所有指定属性”的类型。一个记录类型的值，必然拥有其定义中的所有字段。与元组最大的不同在于，记录的字段是具名的 (named)，这使得代码具有很强的自描述性和可读性

2.8.1 基本语法与使用：

定义：使用 `type` 关键字，并用花括号 `{}` 将所有字段包围起来。每个字段都有一个明确的名称和类型，用分号 `;` 分隔

```
1 (* 定义一个学生记录类型 *)  
2 type student = {  
3   id : int;  
4   name : string;  
5   gpa : float;  
6 }
```

创建记录值：同样使用花括号 `{}`，并通过字段名给每个字段赋值

```
1 let alice = {  
2   id = 101;  
3   name = "Alice";  
4   gpa = 3.8;  
5 }
```

访问记录数据：

- 使用点操作符 `.` 来访问特定字段的值。

```
1 let student_name = alice.name (* "Alice" *)
```

- 使用 `let` 绑定进行解构，一次性提取多个字段。

```
1 let { id; name } = alice (* id 为 101, name 为 "Alice" *)
```

2.8.2 记录的更新

函数式更新 (`with`)

- OCaml 的记录默认是**不可变的 (immutable)**。修改记录的推荐做法是使用 `with` 关键字创建一个新的、只有部分字段不同的副本，而原始记录保持不变。

```
1 let updated_alice = { alice with gpa = 4.0 }  
2 (* updated_alice 是一个全新的记录: { id = 101; name = "Alice";  
   gpa = 4.0 } *)  
3 (* 原始的 alice 记录保持不变: { id = 101; name = "Alice"; gpa =  
   3.8 } *)
```

原地修改 (`mutable`)

- 在少数需要高性能或模拟命令式状态的场景下，可以在定义记录类型时，用 `mutable` 关键字让某个字段变得可变
- 使用 `<-` 操作符来直接修改可变字段的值

```
1 type character = {  
2   name: string;  
3   mutable hp: int; (* HP 是可变的 *)  
4 }  
5 let aragorn = { name = "Aragorn"; hp = 100 }  
6 aragorn.hp <- aragorn.hp - 20 (* 直接在原记录上修改, 现在  
   aragorn.hp 是 80 *)
```

2.8.3 记录双关语 (Record Punning)

- 如果一个字段名和你希望绑定的变量名完全相同, 在创建或解构记录时可以省略字段赋值, 让代码更简洁

```
1 let id = 102  
2 let name = "Bob"  
3 let gpa = 3.5  
4 (* 常规写法 *)  
5 let bob_long = { id = id; name = name; gpa = gpa }  
6 (* Punning 写法 *)  
7 let bob_short = { id; name; gpa }
```

2.8.4 核心应用场景

1. **为程序核心实体建模**: 当你需要表示一个拥有多个固定属性的事物时 (如用户、产品、配置项), 记录是最佳选择, 因为它提供了清晰的字段名, 增强了代码的可读性和可维护性。
2. **向函数传递一组复杂的参数**: 将多个参数打包到一个记录中, 可以让函数签名更简洁, 并且调用时不易出错, 因为每个参数都有明确的标签。

2.9 元组、变体、记录的对比

特性	元组 (Tuple)	变体 (Variant)	记录 (Record)
核心理念	“和”逻辑 (AND) 多个值必须同时存在	“或”逻辑 (OR) 只能是多种可能性中的一种	“和”逻辑 (AND) 多个值必须同时存在
数据表示	匿名的、有序的集合。 通过位置区分。	带标签的、分类的数据。 通过构造器区分。	带名称的、自描述的属性集。 通过字段名区分。
访问方式	按位置解构 <code>let (a, b) = ...</code>	必须用模式匹配 <code>match ... with ...</code>	按名称访问 <code>my_record.name</code>
最佳用途	从函数返回多个值。	表示多种可能的状态或结果 (如 <code>option</code> 类型, AST 节点)。	为程序中的核心“事物” (如用户、产品) 建模。

2.10 引用

理解：`ref` 是一个标准库提供的、用来封装可变数据的“盒子”

语法：

- 创建: `let my_ref = ref 初始值`
- 读取: `!my_ref` (解引用, 读作 "bang")
- 写入: `my_ref := 新值`

```

1 | let counter = ref 0
2 | counter := !counter + 1
3 | (* !counter 的值现在是 1 *)

```

核心理念：共享的可变状态

- 传递一个 `ref` (盒子本身) 给函数时, 函数可以修改盒子里的内容, 并且这个修改对函数外部是可见的

- `ref` 可以理解为一个只有一个 `mutable` 字段的记录的语法糖
 - `ref 10`等价于`{ mutable contents = 10 }`
 - `!r`等价于`r.contents`
 - `r := 20`等价于`r.contents <- 20`

关键特性：共享与副作用

`ref` 的核心特性在于其**共享的可变状态**。当你传递一个 `ref` 本身（盒子）时，任何持有该引用的代码都可以修改盒内的同一个值，这个修改对所有其他持有者都可见。这种行为被称为**副作用 (Side Effect)**，因为它改变了函数外部的状态。

使用场景：

- **实现循环计数器**：当需要用递归模拟一个需要计数或索引变化的循环时
- **累积结果**：当需要一个在多次函数调用或递归中不断更新的累加器，又不想通过函数参数层层传递时

2.11 异常

理解：OCaml 中处理**意外或错误情况**的机制。会**中断**程序的正常执行流程，并立即将控制权“抛”给上层的错误处理器

核心语法：

- **定义异常**：`exception MyError [of <type>]`
 - 本质是定义一个新的、特殊的变体构造器。可以不带参数，也可以携带一个值。
- **抛出异常**：`raise <exception_value>`
- **捕获异常**：`try <expression> with <pattern_matching>`

```

1 (* 1. 定义一个可以携带错误信息的异常 *)
2 exception CalculationError of string
3
4 let divide x y =
5   if y = 0 then
6     (* 2. 抛出异常 *)
7     raise (CalculationError "Division by zero")

```

```

8   else
9       x / y
10
11 let () =
12     try
13         (* 3. 尝试执行可能出错的代码 *)
14         let result = divide 10 0 in
15         Printf.printf "Result: %d\n" result
16     with
17     | CalculationError msg -> (* 捕获特定异常并处理 *)
18         Printf.eprintf "Error: %s\n" msg

```

2.12 模块

2.12.1 核心定义：模块 (Module) 与 签名 (Signature)

- 模块系统的基石，两者是一体两面的关系。
- 模块 (`module ... = struct ... end` 或 `.ml` 文件)
 - 是什么：一个包含了一组相关代码（如 `let` 绑定、`type` 定义等）的集合。
 - 目的：组织代码。一个 `.ml` 文件本身就是一个模块。
- 签名 (`module type ... = sig ... end` 或 `.mli` 文件)
 - 是什么：模块的“接口”或“说明书”。它只声明模块对外暴露了哪些东西（值、类型等），但隐藏了具体的实现细节。
 - 目的：信息隐藏和抽象。这是模块系统最重要的功能。

```

1  (* 1. 定义一个“堆栈”的接口 *)
2  module type STACK = sig
3      type 'a t          (* 这是一个抽象类型，外部不知道它是什么 *)
4      val push : 'a -> 'a t -> 'a t
5      val pop  : 'a t -> 'a t
6      exception Empty
7  end
8
9  (* 2. 实现这个接口 *)
10 module MyStack : STACK = struct
11     type 'a t = 'a list (* 在内部，我们用 list 实现，但外部不知道 *)

```

```
12 | let push = List.cons
13 | let pop = List.tl
14 | exception Empty
15 | end
```

2.12.2 如何使用模块

- 访问 (`ModuleName.item`)
 - 最基本的访问方式，通过点操作符访问模块内公开的成员。如 `MyStack.push`。
- 模块别名 (Module Aliasing)
 - 可以给一个已存在的模块起一个更短或更具描述性的别名。
 - 语法: `module NewName = ExistingModuleName`
 - 示例: `module L = List`，之后就可以用 `L.map` 代替 `List.map`。
- 打开 (`open ModuleName`)
 - 作用：将一个模块的公开成员导入到当前作用域，从而可以省略模块名前缀直接使用。
 - 局部打开 (Local Open): `let open String in ...` 是一个更安全的方式，它只在 `in` 后面的局部作用域内打开模块，避免污染全局命名空间。
- 包含 (`include ModuleName`)
 - 作用：将一个模块的所有内容（包括实现）“复制粘贴”到当前模块中，使其成为当前模块的直接成员。这是一种实现代码复用和组合的方式，类似于“混入 (mixin)”。

`open` vs. `include` 核心区别:

特性	<code>open ModuleName</code>	<code>include ModuleName</code>
核心比喻	抄近路	兼并融合
作用	引入命名空间，让你可以直接访问成员，不改变当前模块的结构。	将目标模块的内容复制到当前模块，会实际地改变和扩展当前模块的结构。
冲突处理	不会产生冲突，只是简化了访问。	如果被包含的模块有同名成员，它会覆盖当前模块的成员。

2.12.3 函子 (Functor) —— 模块的“函数”

核心定义: 函子是一个接收模块作为参数，并返回一个新模块的“超级函数”。

目的: 实现泛型编程和大规模代码复用。

```
1 (* 1. 定义一个符合 Map.Make 要求的“键”模块 *)
2 module IntKey = struct
3   type t = int
4   let compare = compare (* 使用内置的 compare 函数 *)
5 end
6 (* 2. 将这个模块作为参数，传给 Map.Make 函子 *)
7 module IntMap = Map.Make(IntKey)
8 (* 3. 得到一个专门处理 int->'a 映射的新模块 IntMap *)
9 let my_map = IntMap.empty
10 let updated_map = IntMap.add 1 "one" my_map
```

2.13 类

2.13.1. 核心定义：类 (Blueprint) vs. 对象 (Instance)

- 类 (**class**): 定义一个“事物”的**蓝图或模板**。它规定了这个事物应该有哪些**属性 (val)** 和**行为 (method)**。
- 对象 (**object**): 通过 **new** 关键字, 根据类的蓝图**创建出来的具体实例**。你可以根据一张蓝图创建出无数个独立的对象, 每个对象都有自己**独立的状态**。

与模块的核心区别:

- 模块是**静态的、唯一的**。你的程序里只有一个 **List** 模块。
- 类是**动态的、可实例化的**。你可以 **new** 出成千上万个 **counter** 对象。

2.13.2 核心语法与对象创建

构造	语法	解释
定义类	<code>class name params = object ... end</code>	定义一个带初始化参数的类。
定义属性	<code>val field = ... val mutable field = ...</code>	定义不可变或可变的 状态 。
定义方法	<code>method meth ... = ...</code>	定义对象的 行为 。
自我引用	<code>object(self) ...</code>	使用 self 在方法内部调用该对象的其他方法。
创建实例	<code>let obj = new class_name ...</code>	使用 new 关键字实例化一个对象。
调用方法	<code>obj#method_name</code>	使用井号 # 调用对象的方法。

```

1 class counter initial_value =
2   object (self)
3     val mutable value = initial_value (* 1. 定义一个可变属性 *)
4

```

```

5      method increment = value <- value + 1 (* 2. 定义一个方法来修改它
*)
6      method get = value
7      method print = Printf.printf "Current value: %d\n" self#get
8  end
9
10 let c1 = new counter 0 (* 3. 创建第一个对象, 初始值为 0 *)
11 let c2 = new counter 100 (* 4. 创建第二个独立的对象, 初始值为 100 *)
12
13 let () =
14     c1#increment;
15     (* 此刻, c1 的 value 是 1, 而 c2 的 value 仍然是 100 *)
16     Printf.printf "c1: %d, c2: %d\n" c1#get c2#get

```

立即对象 (Immediate Object)

也可以不通过 `class` 直接创建一个唯一的对象。直接对象不能被继承。

```

1 let immediate_obj =
2     object
3       val msg = "I am a unique object"
4       method greet = print_endline msg
5     end
6 (* 直接调用: immediate_obj#greet *)

```

2.13.3 多态类 (Polymorphic Classes)

通过在类名前使用 `['a]`, 可以创建能处理任何类型的“泛型”类。

```

1 class ['a] stack =
2     object
3       val mutable list : 'a list = []
4
5       method push x = list <- x :: list
6       method pop =
7         let hd = List.hd list in
8         list <- List.tl list;
9         hd
10      method is_empty = (list = [])

```

```

11     end
12
13 let int_stack = new stack (* OCaml 推断出这是一个 int stack *)
14 let () = int_stack#push 10
15
16 let string_stack : string stack = new stack (* 也可以显式指定类型 *)
17 let () = string_stack#push "hello"

```

2.13.4 继承

继承是 OOP 中实现代码复用和扩展的主要方式。

- `inherit parent_class`
 - 作用: 子类继承父类的所有属性和方法。
- 方法重写 (Overriding)
 - 作用: 在子类中定义一个与父类同名的方法，即可覆盖父类的行为。
- 调用父类方法 (`as super`)
 - 作用: 如果你想在重写的方法中调用父类的原始版本，可以在继承时给父类起一个别名。这是一个常见模式。

```

1 class parent =
2   object
3     method greet = "Hello"
4   end
5
6 class child =
7   object
8     inherit parent as super (* 1. 继承并给父类起别名 super *)
9     (* 2. 重写 greet 方法 *)
10    method! greet = (super#greet) ^ ", World!" (* 3. 通过 super 调用父类方法 *)
11  end

```

2.13.5 抽象与封装

- 类接口 (`class type`)

- 类似于模块的签名，用于定义一个类对外暴露的公共接口，隐藏内部细节

```
1 class type greeter_interface =  
2   object  
3     method greet : string  
4   end  
5  
6 (* 使用接口限制对象的类型 *)  
7 let greet (p : greeter_interface) = print_endline p#greet
```

- 私有方法 (`private method`)
 - 定义一个只能在对象内部通过 `self#...` 调用的私有方法。外部无法访问，用于隐藏实现细节
- 抽象 (`virtual`)
 - 定义一个没有实现的“抽象方法”，必须由子类来提供实现。包含 `virtual` 方法的类本身也必须被标记为 `virtual`，且不能被直接 `new`
- 类型转换 (`::>`)
 - 将一个子类的对象**显式地**转换成其父类的类型（向上转型）。OCaml 不会自动进行这种转换
 - `let p : parent = (new child ::> parent)`

2.14 模块和类的对比

2.14.1 使用方式上的对比

Module使用规则	Class使用规则
module type 和 sig-end 配合使用	class type和object配合使用
module 和 struct-end 配合使用， 但是可以直接module MyList = List用于重命名模块	class和object配合使用
用open和include可以类似继承，前者只是改变域名解析，后者展开全部模块代码，二者作用域也不相同	用inherit实现继承，一般为区分 子类和父类，用inherit father as super表示区分
val定义变量，不允许mutable，允许 ref	val定义变量，允许mutable，不 推荐ref
val声明方法，let定义方法	method声明、定义方法
module名大写首字母	class名小写首字母
变量默认是公开的，可以通过md.xxx	变量默认是私有的，只有方法公开

2.14.2 对模块和类特性和作用的理解

	模块 (Module)	类 (Class)
核心区别	静态架构工具 用于组织程序。	动态实体模板 用于创建对象。
生命周期	编译期 在编译时确定结构。	运行期 在运行时创建实例。
实例数量	唯一单例 程序中只有一个该模块。	多个实例 可以 <code>new</code> 出无数个对象。
主要用途	搭建程序的宏观框架。	建模程序的微观实体。

2.14.3 为什么编译器项目用模块而不是类

编译器项目是一个完美的静态流水线 (Static Pipeline):

源代码字符串 -> [Lexer模块] -> Token列表 -> [Parser模块] -> AST -> [Evaluator模块] -> 最终结果

流程中的每一个环节 (Lexer, Parser, Evaluator) 都是一个功能明确、独立且唯一的处理站。不需要有多个实例，也不需要维护复杂多变的状态。它们只需要一个清晰的接口，而这正是 OCaml 模块系统的用武之地。它能够构建出宏观上清晰、组件间解耦、接口稳定的软件架构。如果用类来构建，会得到一个奇怪的设计：有一个 `Parser` 对象，但永远只会 `new Parser()` 一次，这完全浪费了类的实例化能力。因此，对于编译器这种本身就是静态结构的应用来说，模块是远比类更自然、更强大的工具。

3 Dune项目

3.1 dune项目结构

```
1  → compiler tree
2  ├── _build
3  |   ├── default
4  |   |   ├── bin
5  |   |   |   ├── main.exe
6  |   |   |   ├── main.ml
7  |   |   |   └── main.mli
8  |   |   ├── compiler.dune-package
9  |   |   ├── compiler.install
10 |   |   ├── compiler.opam
11 |   |   ├── lib
12 |   |   |   ├── ast.ml
13 |   |   |   ├── compiler.a
14 |   |   |   ├── compiler.cma
15 |   |   |   ├── compiler.cmxa
16 |   |   |   ├── compiler.cmxs
17 |   |   |   ├── compiler.ml-gen
18 |   |   |   ├── lexer.ml
19 |   |   |   ├── parser.ml
20 |   |   |   └── parser.mli
21 |   |   ├── META.compiler
22 |   |   └── test
23 |   |       ├── test_compiler.exe
24 |   |       └── test_compiler.ml
```

```

25 | |      └─ test_compiler.mli
26 | |      └─ install
27 | |      └─ default
28 | |      └─ bin
29 | |      |   └─ compiler -> ../../../../default/bin/main.exe
30 | |      └─ lib
31 | |      └─ compiler
32 | |      └─ dune-package ->
    | |      ../../../../default/compiler.dune-package
33 | |      └─ META -> ../../../../default/META.compiler
34 | |      └─ opam -> ../../../../default/compiler.opam
35 |   └─ log
36 └─ bin
37 |   └─ dune
38 |   └─ main.ml
39 └─ compiler.opam
40 └─ dune-project
41 └─ lib
42 |   └─ ast.ml
43 |   └─ dune
44 |   └─ lexer.ml
45 |   └─ lexer.mll
46 |   └─ parser.ml
47 |   └─ parser.mli
48 |   └─ parser.mly
49 └─ test
50 |   └─ dune
51 |   └─ test_compiler.ml
52
53 14 directories, 37 files

```

3.1.1 源代码目录

编写和修改代码的核心区域。

- `lib/`
 - 作用: 存放项目的核心逻辑, 作为一个可复用的“库”。这里包含了编译器的所有核心组件
 - `ast.ml`: 定义抽象语法树 (AST) 的类型

- `lexer.mll`: `ocamllex` 的源文件，定义了词法规则，用于生成 `lexer.ml`
- `parser.mly`: `ocamlyacc` 的源文件，定义了语法规则，用于生成 `parser.ml` 和 `parser.mli`
- `dune`: 配置文件，告诉 Dune 如何将这个目录下的所有 `.ml` 文件编译成一个名为 `compiler` 的库

注意：如果将 `ocamllex` 和 `ocamlyacc` 的生成逻辑放在 `dune` 中，而不是在终端手动执行 `ocamllex` 和 `ocamlyacc` 去生成 `lexer.ml`、`parser.ml` 和 `parser.mli`，则这些文件会被放到 `_build` 目录下面，不会出现在 `lib` 目录下。

考试时给的方式大概率是放到 `_build` 目录下面的。因此不要手动执行 `ocamllex` 和 `ocamlyacc` 去生成 `lexer.ml`、`parser.ml` 和 `parser.mli`，也不要代码中随便 `open Parser` 等模块，不要随便在 `dune` 文件的配置中添加这些模块，否则会报错。

- `bin/`
 - 作用: 存放项目的“入口”或“驱动程序”，即最终的可执行文件
 - `main.ml`: 程序的主函数。主要工作是调用 `lib/` 目录中定义的库函数来执行编译任务
 - `dune`: 配置文件，告诉 Dune 如何编译 `main.ml`，并链接到 `lib/` 中定义的 `compiler` 库，最终生成一个名为 `main.exe` 的可执行文件
- `test/`
 - 作用: 存放测试代码。
 - `test_compiler.ml`: 包含了用于测试 `compiler` 库功能的代码
 - `dune`: 配置文件，告诉 Dune 如何编译和运行测试

3.1.2 配置与生成文件

- `dune-project`
 - 作用: 项目的根配置文件。定义了整个项目的名称、使用的 Dune 语法版本等全局信息。每个 Dune 项目必须有且只有一个此文件
- `compiler.opam`

- 作用: OCaml 包管理器 (OPAM) 的元数据文件。如果想将项目发布为一个可以被其他人安装的包, 就需要这个文件。对于课程项目, 可以暂时忽略
- `_build/`
 - 作用: Dune 的工作目录, 所有编译产物都存放在这里。最好不要手动修改这个目录里的任何东西
 - `_build/default/`: 默认的编译环境
 - `_build/default/lib/`: 存放 `lib` 库编译后的文件 (如 `.cmo`, `.cmi`, `.cmx`)
 - `_build/default/bin/`: 存放 `bin` 目录编译后的可执行文件 (如 `main.exe`)
 - `_build/default/test/`: 存放测试编译后的文件

3.2 dune配置文件

3.2.1 dune-project

`dune-project` 文件位于项目的根目录下, 用于定义整个项目的元数据和全局配置

`(lang dune X.Y)`: 指定当前项目使用的 Dune 语言版本

`(name project_name)`: 定义项目的名称

3.2.2 lib下的dune

编译该目录下的 OCaml 源文件为一个可被其他模块或可执行文件使用的库

```
1 (library
2   (name <library_name>)
3   (modules <module1> <module2> ...)
4   (libraries <dependency1> <dependency2> ...)
5   ; ... 其他可选字段 ...
6 )
```

`name <library_name>:`

- 定义库的名称。其他模块或可执行文件在 `(libraries ...)` 字段中引用这个库时, 就使用这个名称。
- 示例: `(name lex_project)`

`modules <module1> <module2> ...:`

- 列出构成这个库的 OCaml 模块 (`.ml` 或 `.mli` 文件, 不带后缀名), Dune 会自动编译这些模块, 并将它们打包到 `<library_name>` 库中
- 示例: `(modules lexer ast parser)`

`libraries <dependency1> <dependency2> ...`

- 列出这个库所依赖的其他 Dune 库

3.2.3 bin下的dune

编译该目录下的 OCaml 源文件为一个独立的可执行程序

```
1 (executable
2   (name <executable_name>)
3   (public_name <public_name>) ; 可选, 用于安装或外部引用
4   (modules <main_module>) ; 通常只有一个入口模块
5   (libraries <dependency1> <dependency2> ...)
6   ; ... 其他可选字段 ...
7 )
```

`name <executable_name>`

- 定义这个可执行文件的名称, 在命令行中使用 `dune exec <executable_name>` 或直接运行 `_build/default/bin/<executable_name>` 时, 会用到这个名称
- 示例: `(name main)`

`public_name <public_name>` (可选)

- 当希望将这个可执行文件安装到系统路径或作为全局命令时使用
- 示例: `(public_name my_compiler)`

`modules <main_module>` (可选):

- 含义: 列出组成这个可执行文件的模块。通常只有一个入口模块（例如 `main.ml` 对应的 `main`）。如果只有一个模块，可以省略此字段

`libraries <dependency1> <dependency2> ...:`

- 含义: 列出这个可执行文件所依赖的 Dune 库。
- 示例: `(libraries lex_project)`
- 作用: 告诉 Dune 在编译此可执行文件时，需要链接这些库

3.3 Dune相关命令操作

- `dune init` (项目初始化)
 - 用于快速创建新的 Dune 项目结构
 - `dune init <project_type> <project_name>`
 - `<project_type>`: 如 `proj` (普通项目), `lib` (库), `exe` (可执行文件)。
`<project_name>`: 新项目的名称
- `dune clean` (清理构建产物)
 - 删除所有由 Dune 生成的构建文件和目录（例如 `_build` 目录），用于清理项目，解决一些奇怪的构建问题
 - `dune clean`
- `dune build` (编译项目)
 - `dune build [目标]`
 - 如果省略 `[目标]`，Dune 会编译 `dune` 文件中定义的所有默认目标。可以指定某个特定的可执行文件或库来编译，例如 `@exe/my_program.exe`。
 - 如果出现“Try to run `dune build`”类似的报错，先运行 `dune build`，然后使用 `Ctrl + Shift + P` 运行 `OCaml: Restart language server` 来解决
- `dune exec` (执行编译后的程序)
 - `dune exec <executable_name> [可选参数...]`
 - `<executable_name>`: 在 `dune` 文件中定义的 `(executable (name ...))`
 - 如果 `dune exec <project_name>` 卡住:直接运行 `_build` 目录下编译好的可执行文件
- `dune utop` (启动 Utop REPL)

- 在项目环境中启动 `utop` (OCaml 的高级 REPL)，可以加载当前项目的模块，方便交互式测试和调试
- `dune utop [模块名]`

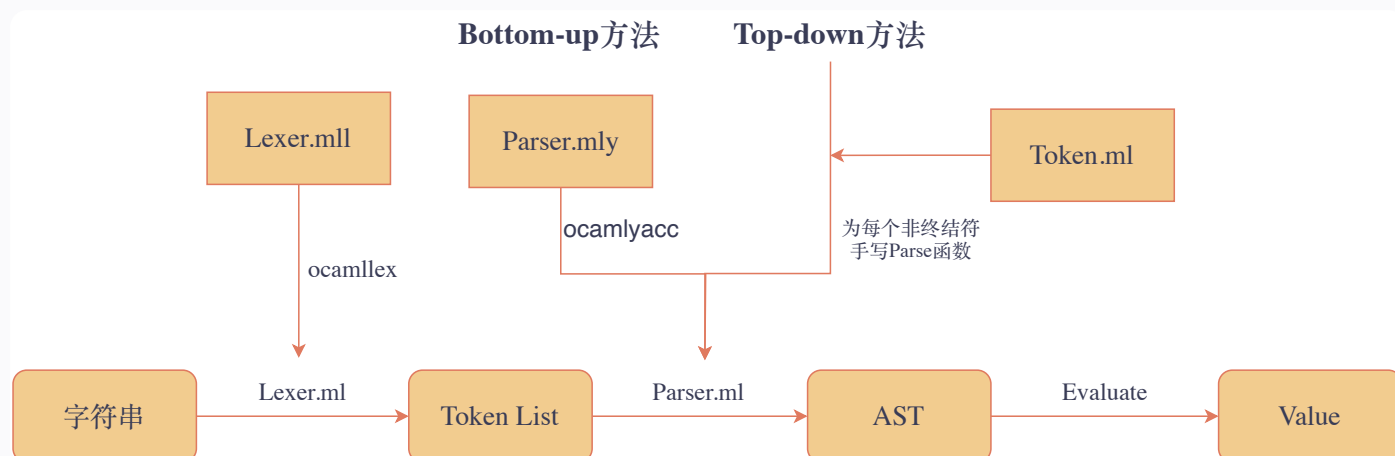
4 编译器前端

4.1 编译器前端的任务

编译器前端的核心任务：

将纯文本的源代码字符串，转换为结构化的抽象语法树（AST）。这个过程主要由词法分析器（Lexer）和语法分析器（Parser）协作完成。

4.1.1 前端工作流程



4.1.2 Top-down和Bottom-up方法

	自顶向下 (Top-Down) 手写递归下降解析器	自底向上 (Bottom-Up) 使用 <code>ocamlyacc</code> 工具
核 心 思 想	预测与推导。 从文法的开始符号出发，尝试推导出与输入匹配的句子。为每个非终结符编写一个解析函数。	移入与规约 (Shift-Reduce)。 不断将 Token 移入栈中，当栈顶形成一个产生式的右部时，将其规约为对应的非终结符。
实 现 方 式	手动编码 (.ml 文件)。 需要为每个非终结符手写一个 <code>parse_...</code> 函数。	自动生成 (.mly 文件)。 只需定义语法规则，<code>ocamlyacc</code> 会自动生成解析器代码。
文 法 要 求	更严格。 文法不能有左递归，且每个产生式的首符集不能交叠，以避免回溯。	更宽松。 可以处理左递归文法，并且通过 <code>%left</code>, <code>%right</code> 等声明可以轻松解决歧义。
优 势	– 灵活性高：可以实现复杂的、上下文相关的错误恢复和报告。 – 直观易懂：代码逻辑直接对应文法推导过程。	– 开发效率极高：处理复杂的算术表达式文法时非常方便。 – 功能强大：内置了解决常见语法歧义（如运算符优先级）的机制。
劣 势	– 开发量大：需要手写大量模板化的代码。 – 文法限制多：改造文法以消除左递归可能很繁琐。	– 灵活性较低：错误报告信息通常比较通用，不易定制。 – 学习曲线陡峭：需要理解 <code>yacc</code> 的工作原理和声明语法。

4.2 AST与文法

4.2.1 AST的概念

抽象语法树 (Abstract Syntax Tree, AST) 是源代码语法结构的树状表示。忽略了源码中不影响语义的细节（如括号、分号、语法辅助规则），只保留了程序的核心逻辑结构。在 OCaml 中，AST 通常用携带参数的递归变体来定义。

```
1 type expr =  
2   | Int of int  
3   | Add of expr * expr  
4   | Mul of expr * expr
```

4.2.2 如何由文法设计AST?

- 产出 `INT`、`ID` 这种基础值的规则->设计一个容器构造器，只携带基础类型的参数（找测试用例的表达式也行）
- 用 `+`、`*`、`if` 这种组合其他部分的规则->设计一个连接件构造器(递归)，用元组的形式携带递归参数

```
1 终结符: INT, +, *  
2 非终结符: Expr, Term, Factor  
3 文法规则:  
4 `Expr` → Expr + Term | Term`  
5 `Term` → Term * Factor | Factor`  
6 `Factor` → INT`
```

为什么 `Expr -> Term` 和 `Term -> Factor` 这两条规则消失了?

这两条规则是纯粹的语法技巧，唯一作用是在 `parser.mly` 里用来定义运算符的优先级，而 AST 的目标是捕捉语义，不是解析过程。

4.2.3 从 AST 到文法

```

1 | type statement =
2 |   | Assign of string * expression (* 连接件:statement -> ID '='
   expression *)
3 |   | Print of expression          (* 连接件:statement -> 'print'
   expression *)
4
5 | and expression =
6 |   | Variable of string          (* 容器:expression -> ID *)
7 |   | Number of int              (* 容器:expression -> INT *)
8 |   | Add of expression * expression (* 连接件:expression ->
   expression '+' expression *)

```

- 只含基础类型的构造器->对应着文法中一条产出终结符的规则
- 递归地包含自身类型的构造器->对应着文法中一条组合其他符号的规则，名字通常包含语义
- 从AST中只能看出部分文法规则，但像括号、分号、结合性优先级规则，则很难推断出来，需要查看Lexer.mll以及Parser.mly（或者手搓的Parser.ml）才能完整的判断出文法。

4.2.4 AST vs. CST (具体语法树)

- 具体语法树 (Concrete Syntax Tree, CST): 也叫分析树 (Parse Tree)。它完全忠实于文法，包含了源代码的所有细节，包括括号、关键字以及所有中间非终结符（如 `Term`，`Factor`）。
- 抽象语法树 (AST): 是 CST 的简化和抽象版本。它丢弃了所有对理解程序语义不必要的“语法噪音”，使得后续的解释或编译过程更加简单。

简单说，**Parser** 在内部构建了 CST，然后将其转换为更简洁的 AST 作为最终输出。

4.3 ocamllex

`ocamllex` 是一个根据词法规则（主要是正则表达式）自动生成 OCaml 词法分析器 (Lexer) 代码的工具。其输入是 `.mll` 文件，输出是 `.ml` 文件

4.3.1 .mll文件的结构

```

1 { header }
2 let ident = regexp ...
3 [refill { refill-handler }]
4 rule entrypoint [arg1... argn] =
5   parse regexp { action }
6   | ...
7   | regexp { action }
8 and entrypoint [arg1... argn] =
9   parse ...
10  and ...
11 { trailer }

```

一个典型的 `.mll` 文件由三个部分组成：

1. { OCaml Header }

- 位于文件顶部，用 `{...}` 包裹。
- 这里的 OCaml 代码会被原样复制到生成的 `.ml` 文件顶部。
- 最常见的用途：`open Parser`，这样 Lexer 就能知道 `Parser` 模块中定义的 `token` 类型（如 `INT`，`PLUS` 等）。

2. Helper Definitions

- 在 Header 之后、`rule` 关键字之前。
- 主要用于 `let ident = regexp ...`，可以为复杂的或常用的正则表达式起一个别名，增加可读性。

```

1 let digit = ['0'-'9']
2 let number = digit+ (* 一个或多个数字 *)

```

3. Lexing Rules

- 文件的主体部分，由一个或多个 `rule` 定义构成，是 `ocamllex` 的核心。

4.3.2 规则区

作用：定义一个函数（`token`），该函数从一个输入流 `lexbuf` 中读取字符，并根据定义的规则，找出能匹配到的最长的字符串（若匹配到多个规则，写在前面的规则优先），返回一个 Token。

规则：

- **最长匹配原则**：匹配尽可能长的字符串。
- **规则优先级**：多条规则都能匹配到同样长度的字符串时，选择写在最前面的那条规则。
- **匹配失败**：没有任何一条规则能够匹配，陷入“卡死”的状态。

语法：

`let ident = regexp`：定义正则表达式别名

`rule entrypoint [arg1... argn] =`：开始定义一个词法分析函数。

- **entrypoint**：词法分析函数名
- `[arg1... argn]`：可选的显式参数，用于维护状态。
- 函数还接收一个隐藏的、自动管理的 `lexbuf` 参数，代表输入流。

`parse regexp as name { action } | ...`：定义具体的匹配规则列表。

- **regexp**：正则表达式，用于匹配输入文本。可以用 `as` 给匹配到的字符串绑定一个变量名，当token带值时常用
- `{ action }`：OCaml 代码。当 **regexp** 成功匹配后，代码会被执行，返回值就是整个词法分析函数的返回值（通常是一个 Token）

`and entrypoint [arg1... argn] =`：定义另一个词法分析函数。

- `and` 关键字允许在同一个 `.mll` 文件中定义多个可以互相调用的词法分析函数。

`[refill { refill-handler }]`：自定义缓冲区填充逻辑。

- 用于自定义当 `lexbuf` 缓冲区里的字符用完，需要从文件或网络等源重新读取数据时的行为。
- 基本不会遇到，可以完全忽略。

```

1 {
2   open Parser
3   exception Error of string
4 }
5
6 rule token = parse
7   | [' ' '\t' '\n'] { token lexbuf }
8   | ['0'-'9']+ as num { INT (int_of_string num) }
9   | '+' { PLUS }
10  | '*' { TIMES }
11  | eof { EOF }
12  | _ as c { raise (Error (Printf.sprintf "Unexpected character:
    %c" c)) }

```

如何填写规则区？

- 明确所有token（一般是终结符+EOF），为所有token写上正则表达式
- 不带值的字符直接返回token名，带值的token要用 `as` 绑定字符，并根据需要进行格式转换
- 添加对空格、换行等的辅助规则
 - `[' ' '\t' '\n'] { token lexbuf }`, 跳过这些字符，再次调用token这个函数（这时必须显式写出 `lexbuf` 这个参数，词法分析器才知道从哪里继续读）
 - `_ as c { raise (Error (Printf.sprintf "Unexpected character: %c" c)) }`, 错误机制，防止匹配失败卡死

4.5 ocaml yacc（Bottom-up方法）

`ocaml yacc` 是一个语法分析器生成器。它读取一个包含语法规则的 `.mly` 文件，并自动生成一个使用自底向上 (Bottom-up) 方法（具体为 LALR(1) 算法）的 OCaml 解析器（`.ml` 和 `.mli` 文件）

4.5.1 .mly文件的结构

```

1 /* parser.mly */
2
3 %{

```

```

4   (* Ocaml代码区 *)
5   open Ast
6 %}
7
8   /* yacc声明区 */
9   %token <int> INT
10  %token PLUS TIMES EOF
11  %left PLUS
12  %left TIMES
13  %start <Ast.expr> main
14
15 %%
16
17 /* 语法规则区 */
18 main:
19     | expr EOF { $1 }
20 ;
21 expr:
22     | term { $1 }
23     | expr PLUS term { Add ($1, $3) }
24 ;

```

一个 `.mly` 文件主要由三个部分组成，由 `%%` 分隔：

1. OCaml 代码区 (Header): 位于文件顶部，用 `%{ ... %}` 包裹。
2. `yacc` 声明区: 位于 `%{...%}` 和 `%%` 之间，用于定义 Token、优先级和入口点。
3. 语法规则区: 位于 `%%` 之后，是文件的核心，定义了语言的文法。

4.5.2 Ocaml代码区

用 `%{ ... %}` 包裹，里面是ocaml的语法，其内容原封不动地复制到 `ocamlyacc` 生成的 `parser.ml` 文件

作用：引入依赖，为后续的“动作”代码准备好必要的模块和函数

一般只有 `open Ast`，不用修改

```

1  %{
2      (* Ocaml代码区 *)
3      open Ast
4  %}

```

4.5.3 yacc声明区

```

1  /* yacc声明区 */
2  %token <int> INT
3  %token PLUS TIMES EOF
4  %left PLUS
5  %left TIMES
6  %start <Ast.expr> main

```

以 `%` 开头, `%%` 结尾, 向 `ocamlyacc` 声明语法分析需要用到的Token和规则

语法:

- `%token` : 声明Token (终结符+EOF)
 - 作用: 列出所有可能从词法分析器 (`lexer.mll`) 传过来的 Token
 - 简单 Token: `%token PLUS TIMES EOF` : 这些 Token 只是符号, 不携带任何额外的值
 - 带值的 Token: `%token <int> INT` : `INT` 这个Token携带 `int` 类型的值

```

1  %token <int> INT
2  %token PLUS TIMES EOF

```

- `%start` 和 `%type` : 确定入口和返回值
 - 作用: 为后续规则的“返回值”指定 OCaml 类型, 以便 OCaml 编译器能进行类型检查
 - `%start <Ast.expr> main`: 组合指令, 包含两个信息:
 - `%start main`: 告诉 `ocamlyacc` 解析过程入口点是 `main` 规则

- `%type <Ast.expr> main`：告诉 `ocamlyacc` 当 `main` 规则解析完毕后，返回一个类型为 `Ast.expr` 的值，这是连接解析过程和 AST 的关键

```
1 | %start <Ast.expr> main
```

- `%left`, `%right`, `%nonassoc`：定义优先级和结合性
 - 优先级：写在后面的行比写在前面的行，优先级更高
 - 结合性：left：左结合、right：右结合，nonassoc：无结合性

```
1 | %left PLUS MINUS    (* 加减法，优先级低 *)
2 | %left TIMES DIV      (* 乘除法，因为在下面，所以优先级高 *)
3 | %right POWER         (* 乘方，优先级最高 *)
```

如何填写声明区？

1. 确定所有 `%token`：找终结符

- 每一个终结符都需要一个对应的 `%token` 声明，且必须全字母大写
- 是否带值：看AST中有基础类型的构造器

2. 确定优先级和结合性 (`%left`, `%right`, ...)

- 找AST中被递归定义的构造器，列出所有这些Token
- 按优先级从低到高排列
- 为每一层选择结合性

3. 确定 `%start` 和 `%type`

- 确定入口点 (`%start`)：找到文法中最顶层的、代表一个完整“程序”或“表达式”的那个非终结符
- 为入口点指定类型：入口点最终必须返回一个完整的 AST。根据 `ast.ml`，确定 AST 主类型
- 为其他非终结符指定类型 (`%type`)：如果非终结符在其动作代码 `{...}` 中返回一个 AST 节点，则它也需要一个类型声明。`ocamlyacc` 可以智能推断，因此这里也可以省略。

4.5.4 语法规则区

```

1  %%
2
3  main:
4      | expr EOF { $1 }
5
6  expr:
7      | term { $1 }
8      | expr PLUS term { Add ($1, $3) }
9
10 term:
11     | factor { $1 }
12     | term TIMES factor { Mul ($1, $3) }
13
14 factor:
15     | INT { Int $1 }

```

`%%` 后的部分都是规则区，此区域定义了语言的具体文法，以及如何根据这些文法构建 AST

语法格式：

```

1  rule_name:
2      | pattern1 { semantic_action1 }
3      | pattern2 { semantic_action2 }
4      ;

```

- **pattern**：一系列由空格隔开的终结符 (Token) 和/或非终结符 (其他规则名)。
- **{ semantic_action }**：一段 OCaml 代码。当一个 **pattern** 被成功匹配时，这段代码会被执行。其返回值将成为该规则 (**rule_name**) 所代表的 OCaml 值 (通常是一个 AST 节点)。
- **\$n**：用于在 **semantic_action** 中获取当前 **pattern** 中第 **n** 个符号所代表的 OCaml 值。例如，在 **expr PLUS term** 中，**\$1** 是 **expr** 的值，**\$2** 是 **PLUS** 的值 (如果它有的话)，**\$3** 是 **term** 的值。

如何填写规则区？

- 填写“main”规则，一般固定

- “照抄文法”：非终结符名字（`Expr`）换成小写（`expr`），把运算符（`+`）换成对应的 Token（`PLUS`）。
- “附加动作”：每条规则模式后面，跟一个 `{...}` 代码块，调用 `ast.ml` 里定义好的构造器（如 `Add`，`Mul`，`Int`），把解析到的部分（通过 `$1`，`$2` 等获取）组装成一个 AST 节点并返回。

4.6 手搓Parser（Top-down方法）

与使用 `ocamlyacc` 自动生成解析器不同，递归下降是一种需要手动编码的自顶向下（Top-Down）解析方法

核心思想：用函数调用栈来模拟推导过程

- 为文法中的每一个非终结符编写一个对应的 `parse_...` 函数。当解析器工作时，它从顶层规则（如 `parse_expr`）开始调用，这些函数根据当前的 Token 预测并选择下一条规则，然后调用其他解析函数。整个过程巧妙地利用了函数调用栈来模拟文法的推导过程

4.6.1 改造文法与定义 Token

1.消除左递归：

递归下降解析器无法处理左递归文法（如 `expr -> expr + term`），因为这会导致 `parse_expr` 无限地调用自身。因此，在编码前必须对文法进行改造。

- 标准改造方法：将左递归规则转换为右递归规则。
 - 原始左递归： $E \rightarrow E \alpha \mid \beta$
 - 改造后： $E \rightarrow \beta E'$ $E' \rightarrow \alpha E' \mid \epsilon$ (ϵ 代表空)

2.手动定义 `token.ml`：

由于不使用 `ocamlyacc`，我们需要手动创建一个 `token.ml` 文件来定义 `token` 类型。这本质上是 `ocamlyacc` 声明区中 `%token` 部分的手动实现

填写方法同yacc声明区：

- 所有终结符+EOF，且最好全字母大写

- 是否带值：看AST中有基础类型的构造器

```
1 | type token =  
2 |   INT of int  
3 |   ADD  
4 |   MUL  
5 |   LPAREN  
6 |   RPAREN  
7 |   EOF
```

4.6.2 实现Parser.ml

编写方法：

- 1.open用到的模块，一般是Ast和token
- 2.根据文法，为每个非终结符编写parse函数，注意互相调用关系
- 3.封装顶层函数供main.ml调用

以下面的文法为例：

```
1 | 终结符： INT n, LPAREN, RPAREN, MUL, ADD, EOF  
2 | 非终结符： expr, expr', term, term', factor  
3 | 语法规则：  
4 |  
5 | expr -> term expr'  
6 | expr' -> ADD term expr' | ε  
7 | term -> factor term'  
8 | term' -> MUL factor term' | ε  
9 | factor -> INT n | LPAREN expr RPAREN
```

非终结符有： `expr, expr', term, term', factor`

使用 `let rec ... and ...` 结构来编写互递归函数，编写顺序无所谓，但从可读性来讲，推荐根据调用文法中调用顺序来编写。

怎么确定每个函数的参数？

- 每个函数必须携带token list，关键是判断哪些函数需要参数ast
- 不带 ast：从左到右，解析出第一个操作数的函数，如 `expr`、`term` 和 `factor`
- 带 ast：接收第一个操作数，然后继续向右看，处理后面跟着的运算符的函数，如 `expr'` 和 `term'`

解析函数的返回值：一个元组：(剩余未处理的 tokens, 解析出的 AST)

怎么编写解析函数的主体部分？

- 本质：用函数调用来模拟推导过程
- 每一条文法规则对应一种调用情况
- 添加错误处理信息

封装顶层调用函数：

- 参数：只有token List
- 返回值：Ast
- 检查残余的token List列表，确保整个输入都被完美地消耗掉了
 - EOF：成功解析，返回Ast
 - 其余情况报错

```
1 open Token
2 open Ast
3
4 let rec parse_expr lst =
5   let t, ast = parse_term lst in
6   parse_expr' t ast
7
8 and parse_term lst =
9   let t, ast = parse_factor lst in
10  parse_term' t ast
11
12 and parse_factor lst =
13   match lst with
14   | INT n :: t -> t, Int n
15   | LPAREN :: t ->
```

```

16     let t, ast = parse_expr t in
17     (match t with
18     | RPAREN :: t -> t, ast
19     | _ -> failwith "Syntax error: expected a closing
parenthesis ')'")
20     | _ -> failwith "Syntax error: expected an integer or an
opening parenthesis '('")
21
22 and parse_expr' lst ast =
23     match lst with
24     | ADD :: t ->
25         let new_t, new_ast = parse_term t in
26         parse_expr' new_t (Add (ast, new_ast))
27     | _ -> lst, ast
28
29 and parse_term' lst ast =
30     match lst with
31     | MUL :: t ->
32         let new_t, new_ast = parse_factor t in
33         parse_term' new_t (Mul (ast, new_ast))
34     | _ -> lst, ast
35
36 let parse tokens =
37     let t, ast = parse_expr tokens in
38     (match t with
39     | [EOF] -> ast
40     | [] -> failwith "Parsing error: unexpected end of input. Did
you forget a closing parenthesis?"
41     | tok::_ ->
42         let error_token_str = Token.string_of_token tok in
43         failwith ("Parsing error: unexpected token: " ^
error_token_str)
44     )
45

```

5 解释器 Interpreter

5.1 解释器的介绍

编译器 vs 解释器

- **编译器 (Compiler):** 翻译源程序（高级语言）到目标程序（低级语言）。目标：更好的性能。
- **解释器 (Interpreter):** 执行源程序，不生成目标程序，直接运行。目标：更容易实现。
- **混合实现:** 如 JVM，先编译成中间代码，再由虚拟机解释执行。即时编译 (JIT) 进一步优化性能。

解释器的工作阶段

- 词法分析 (Lexing) 和 语法分析 (Parsing)
- 语义分析 (Semantic Analysis): 包含类型重建（类型推导和类型检查）
- 求值 (Evaluation): 解释器最核心的部分，在执行 AST 时发生

5.2 求值 (Evaluation)

求值 (Evaluation): 将抽象语法树 (AST) 不断简化，直到它只剩下一个值 (value) 的过程

5.2.1 小步求值 (Small-Step Semantics)

思想: $e \rightarrow e_1 \rightarrow \dots \rightarrow e_n \rightarrow v$ 。它精确地描述了表达式如何进行下一步最微小的归约，整个求值过程就是一系列连续的单步归约。

We establish the following **stepping** rules (we use $\rightarrow$ for a **single-step**):

- **Constants:** Constants (integers) are already *values*, so we **cannot** **step** them further.
- **Binary Operators:** A binary operator application $e_1 \text{ bop } e_2$ has two subexpressions, e_1 and e_2 . Since e_1 and e_2 can also be *values*, we need to consider three different circumstances:

- I. $v_1 \text{ bop } v_2 \rightarrow v$
- II. $v_1 \text{ bop } e_2 \rightarrow v_1 \text{ bop } e_2'$
- III. $e_1 \text{ bop } e_2 \rightarrow e_1' \text{ bop } e_2$

$e_1 \text{ bop } v_2 \rightarrow e_1' \text{ bop } v_2$ indicates that as we **step** the whole expression, we actually recursively **step** e_1 .

You may notice that we don't consider $e_1 \text{ bop } v_2 \rightarrow e_1' \text{ bop } v_2$, because it is included in III.

形式化规则 (Formal Rules) for $e_1 \text{ bop } e_2$:

- **I. 运算规则 (Primitive Operation):** 当两个操作数都已是值时，执行运算。

$v1 \text{ bop } v2 \rightarrow v$

- II. 右侧步进规则 (Step Right): 当左操作数是值, 但右操作数不是时, 步进右操作数。

$e2 \rightarrow e2' \Rightarrow v1 \text{ bop } e2 \rightarrow v1 \text{ bop } e2'$

- III. 左侧步进规则 (Step Left): 当左操作数不是值时, 优先步进左操作数 (最左优先)。

$e1 \rightarrow e1' \Rightarrow e1 \text{ bop } e2 \rightarrow e1' \text{ bop } e2$

实现模式:

is_value 函数: 辅助函数, 检查一个表达式是否已经是值($\text{expr} \rightarrow \text{bool}$)

step 函数: 递归函数, 负责表达式的单步求值

- 如果遇到数字 (**Int**), 则抛出错误, 因为它不能再步进
- 对于二元操作符 (**Binop (binop, e1, e2)**)
 - 如果 **e1** 和 **e2** 都是值, 则调用 **step_binop** 执行实际的原始运算
 - 如果 **e1** 是值, 则对 **e2** 进行一步求值
 - 否则 (最左优先), 对 **e1** 进行一步求值

step_binop 函数: 原始二元操作 $v1 \text{ binop } v2$, 要求 **v1** 和 **v2** 都是值。根据不同操作符 (**Add**, **Sub**, **Mul**, **Div**) 运算, 并处理例如“除零”的错误

eval 函数: 通过递归地调用 **step** 函数, 将表达式 **e** 完全求值到一个值 **v**。它会重复执行 $e \mapsto \text{step } e \mapsto \text{eval } e$ 直到 **e** 成为值

5.2.2 大步求值(Big-Step Semantics)

思想: $e \Rightarrow v$ 。一个表达式 **e** 直接求值得到最终结果 **v** 的关系, 隐藏了所有中间步骤

Now, let's establish rules for **big-step**:

- **Constants**: Constants **big-step** to themselves:

$$i \Rightarrow i$$

- **Binary Operators**: Binary operators just **big-step** both of their subexpressions, then apply whatever the primitive operator is:

```
e1 bop e2 ==> v
  if e1 ==> v1
  and e2 ==> v2
  and v is the result of primitive operation v1 bop v2
```

形式化规则 (Formal Rules):

- 常量规则:

$$i \Rightarrow i$$

- 二元运算规则:

$$\begin{array}{l} e1 \Rightarrow v1 \text{ 且 } e2 \Rightarrow v2 \text{ 且 } v \text{ 是 } v1 \text{ bop } v2 \text{ 的原始运算结果} \Rightarrow \\ e1 \text{ bop } e2 \Rightarrow v \end{array}$$

实现模式:

eval_big 函数: 递归函数, 直接实现了大步求值的思想, 递归地调用自身来求值子表达式, 然后将结果组合起来

- 如果遇到整数 (**Int**), 直接返回自身, 因为它们已经是值
- 对于二元操作符 (**Binop (binop, e1, e2)**), 则调用 **eval_bop** 函数进行处理

eval_bop 函数: 辅助 **eval_big**, 在子表达式求值后, 执行原始的二元运算

5.2.3 对比总结

特性	小步求值 (Small-Step)	大步求值 (Big-Step)
关注点	过程 (How it computes)	结果 (What it computes)
实现方式	迭代/循环 (<code>step</code> 循环)	递归 (<code>eval_big</code> 递归)
适用性	更容易建模复杂的语言特性 (如并发、异常)	更接近真实解释器的直接实现方式

5.3 SimPL

5.3.1 SimPL 语法概览

```
e ::= x | i | b | e1 bop e2
    | if e1 then e2 else e3
    | let x = e1 in e2

bop ::= + | * | <=

x ::= <identifiers>

i ::= <integers>

b ::= true | false
```

SimPL 语言是一个包含核心编程元素的小型语言，其语法构成如下：

- 基本类型: 整数 (`i`) 和布尔值 (`b`, `true` 或 `false`)。
- 变量: 用标识符 (`x`) 表示。
- 二元运算: `e1 bop e2`，其中 `bop` 包括 `+`, `*`, `<=` 等。

- **条件表达式**: `if e1 then e2 else e3`, 当 `e1` 的值为 `true` 时, 表达式结果为 `e2`, 否则为 `e3`。
- **局部变量绑定**: `let x = e1 in e2`, 计算 `e1` 的值, 将结果绑定到变量 `x`, 这个绑定关系在 `e2` 表达式中生效

相比于普通计算器的文法, 最主要的变化就是多了 `if` 和 `let` 两个语句, 因此本部分特别关注这两个表达式的求值。

5.3.2 替换 (Substitution)

替换: 用某个变量对应的值或已求值的表达式来替换该变量, 是解释器在执行过程中处理变量绑定、函数应用或宏展开等基本机制

核心概念: $e'\{e/x\}$, 表示在表达式 `e'` 中, 将所有自由出现的变量 `x` 替换为表达式 `e`

- **subst 函数**:
- **签名**: `expr->expr->string->expr`
- **具体实现**:
 - **常量**: 返回自身
 - **变量**: 同名则替换, 否则不替换
 - $x\{e/x\} = e$
 - $y\{e/x\} = y$
 - **二元/条件表达式**: 递归地对所有子表达式进行替换。
 - $(e1 \text{ bop } e2)\{e/x\} = e1\{e/x\} \text{ bop } e2\{e/x\}$
 - $(\text{if } e1 \text{ then } e2 \text{ else } e3)\{e/x\} = \text{if } e1\{e/x\} \text{ then } e2\{e/x\} \text{ else } e3\{e/x\}$
 - **let 表达式**:
 - **情况一 (变量屏蔽)**: $(\text{let } x = e1 \text{ in } e2)\{e/x\} = \text{let } x = e1\{e/x\} \text{ in } e2$
 - 内部的 `let` 绑定了同名变量 `x`, 它“屏蔽”了外部的 `x`。因此, 替换操作不进入主体 `e2`。
 - **情况二 (常规)**: $(\text{let } y = e1 \text{ in } e2)\{e/x\} = \text{let } y = e1\{e/x\} \text{ in } e2\{e/x\}$ (当 `y` 与 `x` 不同时)

- 替换操作正常递归进入绑定表达式 `e1` 和主体 `e2`。

```
1 let rec subst e v x = match e with
2   | Var y -> if x = y then v else e
3   | Bool _ -> e
4   | Int _ -> e
5   | Binop (binop, e1, e2) -> Binop (binop, subst e1 v x, subst e2
  v x)
6   | If (e1, e2, e3) -> If (subst e1 v x, subst e2 v x, subst e3 v
  x)
7   | Let (y, e1, e2) ->
8     let e1' = subst e1 v x in
9     if x = y
10    then Let (y, e1', e2)
11    else Let (y, e1', subst e2 v x)
```

5.3.3 求值规则 (Evaluation Rules)

`if` 表达式的求值规则

表达式: `if e1 then e2 else e3`

小步求值:

The **small-step** rules you designed for *if* expressions should be like:

- I. `if true then e2 else e3 --> e2`
- II. `if false then e2 else e3 --> e3`
- III. `if e1 then e2 else e3 --> if e1' then e2 else e3`

大步求值：

- The **big-step** rules you designed for *if* expressions should be like:

I. $\text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow v2$

$\text{if } e1 \Rightarrow \text{true}$

and $e2 \Rightarrow v2$

II. $\text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow v3$

$\text{if } e1 \Rightarrow \text{false}$

and $e3 \Rightarrow v3$

	小步求值 ($\rightarrow$)	大步求值 ($\Rightarrow$)
规则 1 (True)	$\text{if true then } e2 \text{ else } e3 \rightarrow e2$	$e1 \Rightarrow \text{true}, e2 \Rightarrow v2 \Rightarrow$ $\text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow v2$
规则 2 (False)	$\text{if false then } e2 \text{ else } e3 \rightarrow e3$	$e1 \Rightarrow \text{false}, e3 \Rightarrow v3 \Rightarrow$ $\text{if } e1 \text{ then } e2 \text{ else } e3 \Rightarrow v3$
规则 3 (Step)	$e1 \rightarrow e1' \Rightarrow$ $\text{if } e1 \text{ then } e2 \text{ else } e3 \rightarrow \text{if } e1' \text{ then } e2 \text{ else } e3$	

- 规则 1 (True):

- 小步: 当条件部分 `e1` 已经被求值为 `true` 时, 整个 `if` 表达式在下一步就直接归约为 `then` 分支 `e2`。
- 大步: 要想让整个 `if` 表达式最终求值为 `v2`, 需要满足两个前提: 首先, 条件 `e1` 必须能最终求值为 `true`; 其次, `then` 分支 `e2` 必须能最终求值为 `v2`。

- 规则 2 (False):

- 小步: 当条件部分 `e1` 已经被求值为 `false` 时, 整个 `if` 表达式在下一步就直接归约为 `else` 分支 `e3`。
- 大步: 逻辑与规则1类似, 只是条件 `e1` 最终求值为 `false`, 且 `else` 分支 `e3` 最终求值为 `v3`。

- 规则 3 (Step):

- 小步: 这是“求值顺序”的核心规则。如果条件 `e1` 本身还是一个复杂的、未被求值的表达式（例如 `1 + 2 <= 4`），那么我们必须先对 `e1` 进行一步归约，得到 `e1'`。整个 `if` 表达式的下一步就变成了 `if e1' then e2 else e3`。 `e2` 和 `e3` 会“等待”，直到 `e1` 被完全求值为 `true` 或 `false`。
- 大步: 大步求值关心的是“最终结果”，不关心中间过程，所以没有与此对应的规则。它会直接“跳到” `e1` 的最终结果。

let 表达式的求值规则

- 小步求值:

Now, we can make some **small-step** evaluation rules for *let* expressions:

- I. `let x = v1 in e2 --> e2{v1/x}`
- II. `let x = e1 in e2 --> let x = e1' in e2`

- 大步求值:

Furthermore, we can make some **big-step** evaluation rules for *let* expressions:

```
let x = e1 in e2 ==> v2
  if e1 ==> v1
  and e2{v1/x} ==> v2
```

	小步求值 ($\rightarrow$)	大步求值 ($\Rightarrow$)
规则 1 (Subst)	<code>let x = v1 in e2 --> e2{v1/x}</code> (当 <code>v1</code> 是一个值时)	<code>e1 ==> v1, e2{v1/x} ==> v2</code> $\Rightarrow$ <code>let x = e1 in e2 ==> v2</code>
规则 2 (Step)	<code>e1 --> e1' $\Rightarrow$</code> <code>let x = e1 in e2 --> let x = e1' in e2</code>	

规则 1 (Subst – 替换):

- **小步:** 当绑定表达式 `e1` 已经被完全求值为一个值 `v1` 时, `let` 表达式的使命就完成了。它的下一步就是执行**替换 (Substitution)**: 将主体 `e2` 中所有自由出现的 `x` 都替换为值 `v1`。
- **大步:** 要让整个 `let` 表达式最终求值为 `v2`, 需要满足两个前提: 首先, 绑定表达式 `e1` 必须能最终求值为一个值 `v1`; 其次, 将 `v1` 替换进 `e2` 后得到的新表达式 `e2{v1/x}`, 必须能最终求值为 `v2`。

规则 2 (Step):

- **小步:** 与 `if` 表达式类似, 这也是一条“求值顺序”规则。如果绑定表达式 `e1` 尚未被求值为一个值, 那么我们必须先对 `e1` 进行一步归约。主体 `e2` 会“等待”, 直到 `e1` 被完全求值为一个值 `v1`, 之后才能触发替换规则。
- **大步:** 不关心这个中间过程。

5.3.4 实现参考

```

1  let is_value : expr -> bool = function
2    | Int _ | Bool _ -> true
3    | Var _ | Binop _ | If _ | Let _ -> false
4
5  (* takes a single step of evaluation of [e] *)
6  let rec step : expr -> expr = function
7    | Int _ | Bool _ -> failwith "Does not step on a number"
8
9    | Var _ -> failwith "Unbound variable"
10
11    (* No need for further stepping if both sides are already
12    values *)
12    | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
13      step_binop binop e1 e2
14
15    (* Evaluate the right side of the binop if the left side is a
16    value *)
16    | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1,
17      step e2)
18
19    (* Leftmost step for binop *)
19    | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
20

```

```

21 | If (Bool true, e2, _) -> e2
22
23 | If (Bool false, _, e3) -> e3
24
25 | If (Int _, _, _) -> failwith "Condition must be a boolean"
26
27 | If (e1, e2, e3) -> If (step e1, e2, e3)
28
29 | Let (x, e1, e2) when is_value e1 -> subst e2 e1 x
30
31 | Let (x, e1, e2) -> Let (x, step e1, e2)
32
33
34 (* implement the primitive operation [v1 binop v2].
35    Requires: [v1] and [v2] are both values. *)
36 and step_binop binop v1 v2 = match binop, v1, v2 with
37 | Add, Int a, Int b -> Int (a + b)
38 | Sub, Int a, Int b -> Int (a - b)
39 | Mul, Int a, Int b -> Int (a * b)
40 | Div, Int a, Int b when b <> 0 -> Int (a / b)
41 | Div, Int _, Int 0 -> failwith "Division by zero"
42 | Leq, Int a, Int b -> Bool (a <= b)
43 | _ -> failwith "Operator and operand type mismatch"
44
45
46 (* fully evaluate [e] to a value [v] *)
47 let rec eval (e : expr) : expr =
48   if is_value e then e else
49     e |> step |> eval
50
51
52 (* interpret [s] by lexing -> parsing -> evaluating and
53    converting the result to a string *)
54 let interp (s : string) : string =
55   s |> parse |> eval |> string_of_expr
56
57 let rec eval_big (e : expr) : expr = match e with
58 | Int _ | Bool _ -> e
59 | Var _ -> failwith "Unbound variable"
60 | Binop (binop, e1, e2) -> eval_binop binop e1 e2
61 | Let (x, e1, e2) -> subst e2 (eval_big e1) x |> eval_big

```

```

62 | If (e1, e2, e3) -> eval_if e1 e2 e3
63
64 and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2
  with
65 | Add, Int a, Int b -> Int (a + b)
66 | Sub, Int a, Int b -> Int (a - b)
67 | Mul, Int a, Int b -> Int (a * b)
68 | Div, Int a, Int b when b <> 0 -> Int (a / b)
69 | Div, Int _, Int 0 -> failwith "Division by zero"
70 | Leq, Int a, Int b -> Bool (a <= b)
71 | _ -> failwith "Operator and operand type mismatch"
72
73 and eval_if e1 e2 e3 = match eval_big e1 with
74 | Bool true -> eval_big e2
75 | Bool false -> eval_big e3
76 | _ -> failwith "Condition must be a boolean"
77
78
79 let interp_big (s : string) : string =
80   s |> parse |> eval_big |> string_of_expr

```

5.4 Lambda Calculus Interpreter

Lambda 演算是计算理论的基石，也是函数式语言的核心。为它实现解释器，关键在于理解其独特的求值语义和变量替换机制

5.4.1 Lambda 演算的语法

一个最小化的 Lambda 演算解释器需要处理三种表达式：

- 变量 (Variable): `x`
- 函数应用 (Application): `e1 e2`
- 匿名函数/抽象 (Abstraction): `fun x -> e`

5.4.2 Lambda 演算的求值语义 (Evaluation Semantics)

对于函数应用 `e1 e2`，主要有两种求值策略，它们的区别在于何时对参数 `e2` 进行求值

求值策略	核心思想	大步求值 (<code>==></code>) 规则
按值调用 (Call-by-Value)	先对参数求值，再进行函数体替换。 这是 OCaml、C 等大多数语言采用的策略。	前提: <code>e1 ==> (fun x -> e)</code> 且 <code>e2 ==> v2</code> 且 <code>e{v2/x} ==> v</code> 结论: <code>e1 e2 ==> v</code>
按名调用 (Call-by-Name)	直接将未求值的参数替换进函数体。 如果参数未被使用，则永远不会被求值，效率更高。	前提: <code>e1 ==> (fun x -> e)</code> 且 <code>e{e2/x} ==> v</code> 结论: <code>e1 e2 ==> v</code>

1. 按值调用 (Call-by-Value)

- 核心思想: “先算好参数，再代入函数”。在执行函数调用之前，必须将要传入的参数表达式 `e2` 完全求值，得到一个最终的值 `v2`。然后，才将这个已经计算好的值 `v2` 替换到函数体 `e` 中进行下一步计算。
- 规则解读: `e1 ==> (fun x -> e)` 并且 `e2 ==> v2` 并且 `e{v2/x} ==> v` $\Rightarrow$ `e1 e2 ==> v`
 - `e1 ==> (fun x -> e)`: 函数部分 `e1` 必须能求值为一个函数 (比如 `fun x -> x + 1`)。
 - `e2 ==> v2`: 这是关键。参数部分 `e2` (比如 `3 + 4`) 必须先被求值为一个最终的值 `v2` (也就是 `7`)。
 - `e{v2/x} ==> v`: 将求值后的参数 `v2` (即 `7`) 替换到函数体 `e` (即 `x + 1`) 中，得到新表达式 `7 + 1`，然后对这个新表达式求值，得到最终结果 `v` (也就是 `8`)。

2. 按名调用 (Call-by-Name)

- 核心思想: “先不算参数，直接把公式代入函数”。在执行函数调用时，不关心参数表达式 `e2` 是什么，直接把它原封不动地替换到函数体 `e` 中。只有在函数体内部真正需要用到这个参数的值时，才会对其进行求值。
- 规则解读: `e1 ==> (fun x -> e)` 并且 `e{e2/x} ==> v` $\Rightarrow$ `e1 e2 ==> v`
 - `e1 ==> (fun x -> e)`: 同样，函数部分 `e1` 必须能求值为一个函数。

- $e\{e2/x\} \implies v$: 这是关键区别。我们直接将未求值的表达式 $e2$ (比如 $3 + 4$) 替换到函数体 e (比如 $x + 1$) 中, 得到新表达式 $(3 + 4) + 1$, 然后才对这个整体进行求值。
- 优势: 如果函数体中从未使用过其参数, 那么这个参数就永远不会被求值, 从而节省了计算资源。

5.4.3 避免捕捉的替换 (Capture-Avoiding Substitution)

替换 $e\{v/x\}$ 是 Lambda 演算的核心操作, 但简单的文本替换会导致变量捕捉 (Variable Capturing) 的严重问题, 即被替换进来的表达式中的自由变量, 被函数体内的绑定错误地“捕获”, 从而改变了程序的语义。

- 问题示例: 考虑 $(\text{fun } x \rightarrow y)\{x/y\}$ 。这个表达式的意图是将一个返回自由变量 y 的函数, 其中的 y 替换为 x , 结果应该是 $\text{fun } x \rightarrow x$ 。但考虑另一个表达式 $(\text{fun } y \rightarrow x)\{y/x\}$ 。其意图是将 x 替换为 y , 但如果简单替换, 会得到 $\text{fun } y \rightarrow y$, 一个恒等函数。原来的自由变量 x 的信息丢失了, y 被内部的 $\text{fun } y \rightarrow$ 错误地捕获了。
- 解决方案: 避免捕捉的替换 (Capture-Avoiding Substitution), 其规则如下:

1. 基本规则:

- $x\{v/x\} = v$ (变量同名, 替换)
- $y\{v/x\} = y$ (当 y 与 x 不同时)
- $(e1 \ e2)\{v/x\} = (e1\{v/x\}) \ (e2\{v/x\})$ (递归进入应用)

2. 函数替换规则 (核心): 对于 $(\text{fun } y \rightarrow e')\{v/x\}$:

- 情况 1 (变量屏蔽): 如果 $y = x$, 则 x 在 e' 中被屏蔽, 替换操作停止, 不对 e' 进行任何操作。

$$(\text{fun } x \rightarrow e')\{v/x\} = \text{fun } x \rightarrow e'$$

- 情况 2 (安全替换): 如果 $y \neq x$ 并且 y 不在 v 的自由变量集合 $FV(v)$ 中, 则可以安全地将替换递归到函数体 e' 中。

$$(\text{fun } y \rightarrow e')\{v/x\} = \text{fun } y \rightarrow (e'\{v/x\})$$

- 情况 3 (重命名以避免捕捉): 如果 $y \neq x$ 但是 y 在 $FV(v)$ 中, 直接替换会导致 v 中的 y 被捕获。此时必须:
 1. 生成一个全新的、不冲突的变量名 z (例如使用 `gensym` 函数)。
 2. 先将函数体 e' 中的所有 y 都替换为 z (这个过程叫 α -conversion)。
 3. 然后再对新的函数体进行 x 的替换。

```
(fun y -> e'){v/x} = fun z -> (e'{z/y}){v/x}
```

- 自由变量 (Free Variables, FV): 要实现上述规则, 必须能计算一个表达式中的自由变量集合。
 - $FV(x) = \{x\}$
 - $FV(e_1 \ e_2) = FV(e_1) \cup FV(e_2)$
 - $FV(\text{fun } y \rightarrow e) = FV(e) - \{y\}$ (绑定操作会从集合中移除变量)

5.4.4 实现示例

```
1 (* check if an expression is a value (i.e., fully evaluated) *)
2 let is_value : expr -> bool = function
3   | Int _ | Bool _ | Func _ -> true
4   | Var _ | Binop _ | If _ | Let _ | App _ -> false
5
6 module VarSet = Set.Make(String)
7 let singleton = VarSet.singleton
8 let union = VarSet.union
9 let diff = VarSet.diff
10 let mem = VarSet.mem
11
12 (** [fv e] is a set-like list of the free variables of [e]. *)
13 let rec fv : expr -> VarSet.t = function
14   | Var x -> singleton x
15   | App (e1, e2) -> union (fv e1) (fv e2)
16   | Func (x, e) -> diff (fv e) (singleton x)
17
18 (** [gensym ()] is a fresh variable name. *)
19 let gensym =
20   let counter = ref 0 in
21   fun () ->
```



```

22     incr counter; "$x" ^ string_of_int !counter
23
24 (** [replace e y x] is [e] with the name [x] replaced
25     by the name [y] anywhere it occurs. *)
26 let rec replace e y x = match e with
27   | Var z -> if z = x then Var y else e
28   | App (e1, e2) -> App (replace e1 y x, replace e2 y x)
29   | Func (z, e') -> Func ((if z = x then y else z), replace e' y
30   x)
31
32 (** [subst e v x] is [e] with [v] substituted for [x], that
33     is, [e{v/x}]. *)
34 let rec subst e v x = match e with
35   | Var y -> if x = y then v else e
36   | Bool _ -> e
37   | Int _ -> e
38   | Binop (binop, e1, e2) -> Binop (binop, subst e1 v x, subst
39   e2 v x)
40   | If (e1, e2, e3) -> If (subst e1 v x, subst e2 v x, subst e3
41   v x)
42   | Let (y, e1, e2) ->
43     let e1' = subst e1 v x in
44     if x = y
45     then Let (y, e1', e2)
46     else Let (y, e1', subst e2 v x)
47   | App (e1, e2) -> App (subst e1 v x, subst e2 v x)
48   | Func (y, e') ->
49     if x = y then e
50     else if not (mem y (fv v)) then Func (y, subst e' v x)
51     else
52       let fresh = gensym () in
53       let new_body = replace e' y fresh in
54       Func (fresh, subst new_body v x)
55
56 (* takes a single step of evaluation of [e] *)
57 let rec step : expr -> expr = function
58   | Int _ | Bool _ -> failwith "Does not step on a number"
59
60   | Var _ -> failwith "Unbound variable"

```

```

60   (* No need for further stepping if both sides are already
    values *)
61   | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
62     step_binop binop e1 e2
63
64   (* Evaluate the right side of the binop if the left side is a
    value *)
65   | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1,
    step e2)
66
67   (* Leftmost step for binop *)
68   | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
69
70   | If (Bool true, e2, _) -> e2
71
72   | If (Bool false, _, e3) -> e3
73
74   | If (Int _, _, _) -> failwith "Condition must be a boolean"
75
76   | If (e1, e2, e3) -> If (step e1, e2, e3)
77
78   | Let (x, e1, e2) when is_value e1 -> subst e2 e1 x
79
80   | Let (x, e1, e2) -> Let (x, step e1, e2)
81
82
83   (* implement the primitive operation [v1 binop v2].
84      Requires: [v1] and [v2] are both values. *)
85   and step_binop binop v1 v2 = match binop, v1, v2 with
86     | Add, Int a, Int b -> Int (a + b)
87     | Sub, Int a, Int b -> Int (a - b)
88     | Mul, Int a, Int b -> Int (a * b)
89     | Div, Int a, Int b when b <> 0 -> Int (a / b)
90     | Div, Int _, Int 0 -> failwith "Division by zero"
91     | Leq, Int a, Int b -> Bool (a <= b)
92     | _ -> failwith "Operator and operand type mismatch"
93
94
95   (* fully evaluate [e] to a value [v] *)
96   let rec eval (e : expr) : expr =
97     if is_value e then e else
98       e |> step |> eval

```

```

99
100
101 (* interpret [s] by lexing -> parsing -> evaluating and
    converting the result to a string *)
102 let interp (s : string) : string =
103     s |> parse |> eval |> string_of_expr
104
105
106 (* CBV stands for call-by-value, and CBN stands for call-by-
    name.
107     If strategy is set to be CBV, the call-by-value strategy is
    adopted.
108     And vice versa.
109
110     Warning 37 should be disabled for this statement.*)
111 type eval_strategy = CBV | CBN
112 let strategy = CBV
113
114 let rec eval_big (e : expr) : expr = match e with
115     | Int _ | Bool _ -> e
116     | Var _ -> failwith "Unbound variable"
117     | Binop (binop, e1, e2) -> eval_bop binop e1 e2
118     | Let (x, e1, e2) -> subst e2 (eval_big e1) x |> eval_big
119     | If (e1, e2, e3) -> eval_if e1 e2 e3
120     | App (e1, e2) -> eval_app e1 e2
121     | Func _ -> e
122
123 and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2
    with
124     | Add, Int a, Int b -> Int (a + b)
125     | Sub, Int a, Int b -> Int (a - b)
126     | Mul, Int a, Int b -> Int (a * b)
127     | Div, Int a, Int b when b <> 0 -> Int (a / b)
128     | Div, Int _, Int 0 -> failwith "Division by zero"
129     | Leq, Int a, Int b -> Bool (a <= b)
130     | _ -> failwith "Operator and operand type mismatch"
131
132 and eval_if e1 e2 e3 = match eval_big e1 with
133     | Bool true -> eval_big e2
134     | Bool false -> eval_big e3
135     | _ -> failwith "Condition must be a boolean"
136

```

```
137
138 (** [eval_app e1 e2] is the [e] such that [e1 e2 ==> e]. *)
139 and eval_app e1 e2 = match eval_big e1 with
140   | Func (x, e) ->
141     let e2' =
142       match strategy with
143       | CBV -> eval_big e2
144       | CBN -> e2
145     in subst e e2' x |> eval_big
146   | _ -> failwith "Cannot apply non-function"
147
148 let interp_big (s : string) : string =
149   s |> parse |> eval_big |> string_of_expr
```

6 附件

6.1 常见符号总结

符号	名称 / 含义	类别	使用场景与解释	示例代码
;	分号 / 序列点	结构与绑定	按顺序执行两个表达式，但丢弃第一个表达式的结果，只返回第二个。常用于有副作用（如打印）的操作。	<pre>print_endline "Step 1"; print_endline "Step 2"</pre>
;;	双分号 / 顶层结束符	结构与绑定	告诉解释器“这段代码写完了，请执行”。	<pre>let x = 10;; (* 在 utop 中执行 *)</pre>
=	等于号	结构与绑定	1. 绑定: 在 <code>let</code> 或 <code>type</code> 语句中用于命名。 2. 比较: 判断两个值的“内容”是否完全一样（结构相等）。	<pre>let x = 5 (* 绑定 *) if x = 5 then ... (* 比较 *)</pre>

,	逗号	定 结 构 与 绑 定	主要用于分隔 元组 (tuple) 中的元素。	<pre>let my_tuple = (10, "hello", true)</pre>
:	冒号 / 类型 注解	类 型 与 模 块	明确地告诉编译器一个值应该是什么类型。增加代码可读性，帮助调试。	<pre>let x : int = 10</pre>
->	箭头 / 函数 类型	类 型 与 模 块	1. 函数类型 : <code>int -> string</code> 表示接收 <code>int</code> 、返回 <code>string</code> 的函数。 2. 模式匹配分支 : 在 <code>match</code> 中分隔模式和其对应的代码。	<pre>let f (x: int): bool = ... match x with \l 0 -> "zero"</pre>
*	星号 / 元组 类型	类 型 与 模 块	在类型定义中，用 <code>*</code> 来表示 元组类型 。	<pre>let pair : (int * string) = (1, "one")</pre>
::	Cons 操作 符	数 据 结 构	列表构造的核心 。将一个元素添加到列表的头部，效率极高 ($O(1)$)。	<pre>let my_list = 1 :: [2; 3]</pre>
@	At 符 号	数 据 结 构	连接两个列表 ，返回一个新列表。效率较低 ($O(n)$)。	<pre>[1; 2] @ [3; 4]</pre>
!	感叹 号 /	可 变	从一个 <code>ref</code> 类型的“盒子”里取出它	<pre>let r = ref 10</pre>

	解引用	性	所包含的值。	<code>let v = !r</code>
	竖线 / “或”	模式匹配	1. <code>match</code> 分支: 分隔 <code>match</code> 中的不同情况。 2. 变体定义: 分隔变体类型中的不同选项。	<code>match x with \l 1 \l 2 -> "one or two" type t = A \l B</code>
_	下划线 / 通配符	模式匹配	匹配任何东西，但我不在乎它是什么。也用于忽略不使用的变量。	<code>match x with \l Some v -> v \l None -> 0 let _ = some_function ()</code>
#	井号 / 方法调用	面向对象	用于调用一个对象的方法。	<code>my_obj#some_method</code>
<-	左箭头 / 字段赋值	可变性	给记录中 <code>mutable</code> 标记的可变字段赋新值。	<code>let c = { mutable count = 0 } c.count <- 1</code>
:=	冒号等于 / 引用赋值	可变性	给一个 <code>ref</code> 类型的“盒子”赋一个新值。	<code>let r = ref 10 r := 20</code>
>	管道操作符	函数应用	将左侧的值“管道化”地传递给右侧的函数，作为其最后一个参数，增加可读性。	<code>x \l> f \l> g (* 等价于 g(f(x)) *)</code>

6.2 大小写总结

类别	规则	示例	强制性
----	----	----	-----

模块名 (Module Name)	首字母必须大写	<code>String</code> , <code>List</code> , <code>MyParser</code>	语法强制 (否则编译错误)
签名名 (Signature Name)	全部大写 (惯例)	<code>STACK</code> , <code>MAP</code>	社区惯例
变体构造器 (Variant Constructor)	首字母必须大写	<code>Some</code> , <code>None</code> , <code>Int</code> , <code>Add</code>	语法强制
异常构造器 (Exception Constructor)	首字母必须大写	<code>Not_found</code> , <code>Failure</code> , <code>Invalid_argument</code>	语法强制
多态变体标签 (Poly. Variant Tag)	首字母必须大 写, 并用反引号 ` 包裹	<code>`Success</code> , <code>`Error</code> , <code>`Ok</code>	语法强制
变量/函数名 (Variable/Function Name)	首字母必须小写 或下划线	<code>x</code> , <code>calculate_area</code> , <code>_private_var</code>	语法强制
记录字段名 (Record Field Name)	首字母必须小写	<code>id</code> , <code>student_name</code>	语法强制
方法名 (Method Name)	首字母必须小写	<code>get</code> , <code>draw</code> , <code>increment</code>	语法强制
类名 (Class Name)	首字母必须小写	<code>counter</code> , <code>stack</code> , <code>pokemon</code>	语法强制
类型名 (自定义) (Custom Type Name)	全部小写	<code>expr</code> , <code>token</code> , <code>student</code>	社区惯例 (但内置类型如 <code>int</code> , <code>string</code> 强 制小写)
类型变量	小写, 并用单引		

(Type Variable)	号 <code>'</code> 开头	<code>'a</code> , <code>'b</code> , <code>'key</code>	语法强制
Token (in <code>.mly</code>)	全部大写	<code>INT</code> , <code>PLUS</code> , <code>LPAREN</code> , <code>EOF</code>	<code>ocamlyacc</code> 惯例

6.3 常用数据模块库函数速查

6.3.1 List库函数速查表

函数 (Function)	签名 (Signature)	说明 (Description)	示例 (Example)
<code>length</code>	<code>'a list -> int</code>	返回列表长度	<code>List.length [1; 2; 3]</code> returns <code>3</code>
<code>hd</code>	<code>'a list -> 'a</code>	返回列表头元素 (若列表为空则抛出异常)	<code>List.hd ["a"; "b"]</code> returns <code>"a"</code>
<code>tl</code>	<code>'a list -> 'a list</code>	返回除头元素外的列表 (若列表为空则抛出异常)	<code>List.tl [1; 2; 3]</code> returns <code>[2; 3]</code>
<code>nth</code>	<code>'a list -> int -> 'a</code>	返回列表中指定索引的元素 (若越界则抛出异常)	<code>List.nth [10; 20; 30] 1</code> returns <code>20</code>
<code>rev</code>	<code>'a list -> 'a list</code>	返回一个元素顺序颠倒的新列表	<code>List.rev [1; 2; 3]</code> returns <code>[3; 2; 1]</code>
<code>init</code>	<code>int -> (int -> 'a) -> 'a list</code>	根据函数生成一个指定长度的列表	<code>List.init 5 (fun i -> i * i)</code> returns <code>[0; 1; 4; 9; 16]</code>

append 或 @	'a list -> 'a list -> 'a list	拼接两个列表	[1; 2] @ [3; 4] returns [1; 2; 3; 4]
concat	'a list list -> 'a list	将一个列表的列表 “压平”成一个列表	List.concat [[1; 2]; [3]] returns [1; 2; 3]
map	('a -> 'b) -> 'a list -> 'b list	对每个元素应用函 数，返回结果组成 的新列表	List.map (fun x -> x * 2) [1; 2] returns [2; 4]
mapi	(int -> 'a -> 'b) -> 'a list -> 'b list	与 map 类似，但 函数会额外接收元 素索引	List.mapi (fun i x -> x+i) [10; 10] returns [10; 11]
iter	('a -> unit) -> 'a list -> unit	对每个元素执行一 个有副作用的函 数，返回 unit	List.iter print_int [1; 2; 3] prints 123
iteri	(int -> 'a -> unit) -> 'a list -> unit	与 iter 类似， 但函数会额外接收 元素索引	List.iteri (fun i x -> Printf.printf "%d:%d " i x) [5; 6] prints 0:5 1:6
fold_left	('acc -> 'a -> 'acc) -> 'acc - > 'a list -> 'acc	从左到右对列表进 行折叠/累积操作	List.fold_left (+) 0 [1; 2; 3] returns 6
fold_right	('a -> 'acc -> 'acc) -> 'a list -> 'acc -> 'acc	从右到左对列表进 行折叠/累积操作	List.fold_right (fun x acc -> x :: acc) [1; 2; 3] [] returns [1; 2; 3]
filter	('a -> bool) -> 'a list -> 'a list	返回一个只包含满 足条件元素的新列 表	List.filter (fun x -> x mod 2=0) [1; 2; 3; 4] returns [2; 4]

find	<code>('a -> bool) -> 'a list -> 'a</code>	返回第一个满足条件的元素 (若无则抛出异常)	<code>List.find (fun x -> x > 2) [1; 3; 4]</code> returns <code>3</code>
find_opt	<code>('a -> bool) -> 'a list -> 'a option</code>	返回第一个满足条件的元素 <code>Some x</code> ，否则返回 <code>None</code>	<code>List.find_opt (fun x -> x > 10) [1; 2]</code> returns <code>None</code>
exists	<code>('a -> bool) -> 'a list -> bool</code>	检查是否存在满足条件的元素	<code>List.exists ((=) 3) [1; 2; 3]</code> returns <code>true</code>
for_all	<code>('a -> bool) -> 'a list -> bool</code>	检查是否所有元素都满足条件	<code>List.for_all (fun x -> x < 5) [1; 2; 3]</code> returns <code>true</code>
partition	<code>('a -> bool) -> 'a list -> 'a list * 'a list</code>	将列表分割成两个列表：满足条件的和不满足的	<code>List.partition (fun x -> x mod 2=0) [1;2;3;4]</code> returns <code>([2; 4], [1; 3])</code>
combine	<code>'a list -> 'b list -> ('a * 'b) list</code>	将两个列表的对应元素组合成元组列表	<code>List.combine [1; 2] ["a"; "b"]</code> returns <code>[(1, "a"); (2, "b")]</code>
split	<code>('a * 'b) list -> 'a list * 'b list</code>	<code>combine</code> 的逆操作，将元组列表拆分为两个列表	<code>List.split [(1,"a"); (2,"b")]</code> returns <code>([1; 2], ["a"; "b"])</code>
sort	<code>('a -> 'a -> int) -> 'a list -> 'a list</code>	根据比较函数对列表进行排序	<code>List.sort compare [3; 1; 2]</code> returns <code>[1; 2; 3]</code>
	<code>('a -> 'a -></code>		<code>List.sort_uniq</code>

<code>sort_uniq</code>	<code>int) -> 'a list</code> <code>-> 'a list</code>	排序后移除相邻的重复元素	<code>compare [3; 1; 2; 1]</code> returns <code>[1; 2; 3]</code>
------------------------	---	--------------	---

6.3.2 String库函数速查表

函数 (Function)	签名 (Signature)	说明 (Description)	示例 (Example)
<code>length</code>	<code>string -> int</code>	返回字符串的长度	<code>String.length "hello"</code> returns <code>5</code>
<code>get</code>	<code>string -> int -> char</code>	获取指定索引位置的字符 (若越界则抛出异常)	<code>String.get "ocaml" 1</code> returns <code>'c'</code>
<code>make</code>	<code>int -> char -> string</code>	创建一个由重复字符组成的指定长度的字符串	<code>String.make 3 'a'</code> returns <code>"aaa"</code>
<code>sub</code>	<code>string -> int -> int -> string</code>	提取子字符串 (从指定索引开始, 提取指定长度)	<code>String.sub "hello" 1 3</code> returns <code>"ell"</code>
<code>concat</code>	<code>string -> string list -> string</code>	使用指定的分隔符连接一个字符串列表	<code>String.concat ", " ["a"; "b"; "c"]</code> returns <code>"a, b, c"</code>
<code>contains</code>	<code>string -> char -> bool</code>	检查字符串是否包含某个字符	<code>String.contains "hello" 'e'</code> returns <code>true</code>
<code>split_on_char</code>	<code>char -> string -> string list</code>	使用指定字符作为分隔符来拆分字符串	<code>String.split_on_char ',' "a,b,c"</code> returns <code>["a"; "b"; "c"]</code>
<code>trim</code>	<code>string -> string</code>	移除字符串首尾的空白字符 (空格, <code>\n</code> , <code>\t</code> 等)	<code>String.trim " hello "</code> returns <code>"hello"</code>
<code>uppercase_ascii</code>	<code>string -> string</code>	将字符串转换为大写 (仅限 ASCII)	<code>String.uppercase_ascii "Ocaml"</code> returns <code>"OCAML"</code>
<code>lowercase_ascii</code>	<code>string -> string</code>	将字符串转换为小写 (仅限 ASCII)	<code>String.lowercase_ascii "Ocaml"</code> returns <code>"ocaml"</code>

6.3.3 类型转换函数表

函数 (Function)	签名 (Signature)	说明 (Description)	示例 (Example)
<code>string_of_int</code>	<code>int -> string</code>	将整数转换为字符串。	<code>string_of_int 123</code> returns <code>"123"</code>
<code>int_of_string</code>	<code>string -> int</code>	将字符串转换为整数。 注意: 若字符串不是合法的整数格式, 会抛出 <code>Failure</code> 异常。	<code>int_of_string "-45"</code> returns <code>-45</code>
<code>int_of_string_opt</code>	<code>string -> int option</code>	安全地将字符串转换为整数。若成功, 返回 <code>Some int</code> ; 若失败, 返回 <code>None</code> 。	<code>int_of_string_opt "1a"</code> returns <code>None</code>
<code>string_of_float</code>	<code>float -> string</code>	将浮点数转换为字符串。	<code>string_of_float 3.14</code> returns <code>"3.14"</code>
<code>float_of_string</code>	<code>string -> float</code>	将字符串转换为浮点数。 注意: 若格式非法, 会抛出 <code>Failure</code> 异常。	<code>float_of_string "2.718"</code> returns <code>2.718</code>
<code>float_of_string_opt</code>	<code>string -> float option</code>	安全地将字符串转换为浮点数。若成功, 返回 <code>Some float</code> ; 若失败, 返回 <code>None</code> 。	<code>float_of_string_opt "3.14"</code> returns <code>Some 3.14</code>
<code>string_of_bool</code>	<code>bool -> string</code>	将布尔值转换为 <code>"true"</code> 或 <code>"false"</code> 。	<code>string_of_bool true</code> returns <code>"true"</code>
<code>bool_of_string</code>	<code>string -> bool</code>	将 <code>"true"</code> 或 <code>"false"</code> 字符串转换为布尔值。 注意: 对其他任何字符串都会抛出 <code>Invalid_argument</code> 异常。	<code>bool_of_string "false"</code> returns <code>false</code>

6.3.4 Printf函数速查表

函数 (Function)	签名 (Signature)	说明 (Description)	示例 (Example)
<code>printf</code>	<code>('a, out_channel, unit) format -> 'a</code>	打印到标准输出。将格式化字符串和参数打印到控制台。	<code>Printf.printf "Name: %s, Age: %d\n" "Alice" 25</code>
<code>sprintf</code>	<code>('a, unit, string) format -> 'a</code>	返回格式化字符串。不打印，而是返回一个构建好的新字符串。	<code>let s = Printf.sprintf "Value: %.2f" 3.14 returns "Value: 3.14"</code>
<code>eprintf</code>	<code>('a, out_channel, unit) format -> 'a</code>	打印到标准错误流。用于打印错误或调试信息，与标准输出分离。	<code>Printf.eprintf "Error: File not found.\n"</code>

占位符	对应类型
<code>%s</code>	<code>string</code> (字符串)
<code>%d</code>	<code>int</code> (整数)
<code>%f</code>	<code>float</code> (浮点数)
<code>%b</code>	<code>bool</code> (布尔值)
<code>%c</code>	<code>char</code> (字符)
<code>\n</code>	换行符

